RENAISSANCE UNIVERSITY, INDORE
SCHOOL OF COMPUTER SCIENCE

Subject name: Operating System; Class - BCAIII

UNIT -1

INTRODUCTION OF OPERATING SYSTEM

Index

1 Introduction to Operation System:

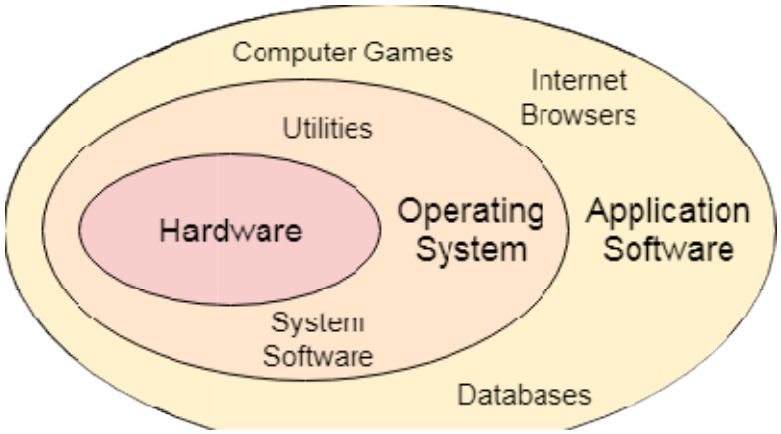
An Operating System acts as a communication bridge (interface) between the user and computer hardware. The purpose of an operating system is to provide a platform on which a user can execute programs in a convenient and efficient manner.

An operating system is a piece of software that manages the allocation of computer hardware. The coordination of the hardware must be appropriate to ensure the correct working of the computer system and to prevent user programs from interfering with the proper working of the system.

Example: Just like a boss gives orders to his employee, in a similar way we request or pass our orders to the Operating System. The main goal of the Operating System is to make the computer environment more convenient to use and Secondary goal is to use the resources in the most efficient manner.

In the Computer System (comprises of Hardware and software), Hardware can only understand machine code (in the form of 0 and 1) which doesn't make any sense to a naive user.

We need a system which can act as an intermediary and manage all the processes and resources present in the system.



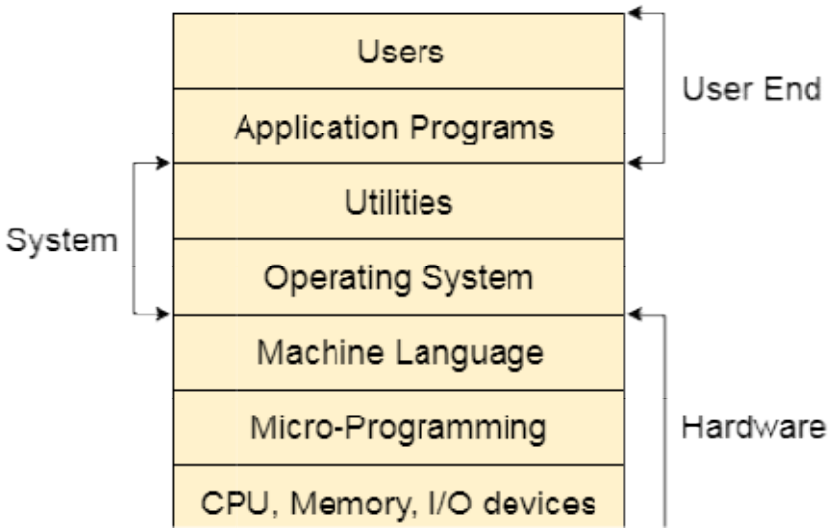
An **Operating System** can be defined as an **interface between user and hardware**. It is responsible for the execution of all the processes, Resource Allocation, CPU management, File Management and many other tasks.

The purpose of an operating system is to provide an environment in which a user can execute programs in convenient and efficient manner.

2 Structure of a Computer System

A Computer System consists of:

- Users-(people who are using the computer)
- Application Programs (Compilers, Databases, Games, Video player, Browsers, etc.)
- System Programs (Shells, Editors, Compilers, etc.)
- Operating System (A special program which acts as an interface between user and hardware)
- Hardware (CPU, Disks, Memory, etc)



3 Important functions of an Operating System:

1. **Security –**
   The operating system uses password protection to protect user data and similar other techniques. it also prevents unauthorized access to programs and user data.

2. **Control over system performance –**
   Monitors overall system health to help improve performance. Records the response time between service requests and system response to having a complete view of the system health. This can help improve performance by providing important information needed to troubleshoot problems.

3. **Job accounting –**
   Operating system Keeps track of time and resources used by various tasks and users, this information can be used to track resource usage for a particular user or group of users.

4. **Error detecting aids –**
   The operating system constantly monitors the system to detect errors and avoid the malfunctioning of a computer system.

5. **Coordination between other software and users** –
   Operating systems also coordinate and assign interpreters, compilers, assemblers, and other software to the various users of the computer systems.

6. **Memory Management –**
   The operating system manages the Primary Memory or Main Memory. Main memory is made up of a large array of bytes or words where each byte or word is assigned a certain address. Main memory is fast storage and it can be accessed directly by the CPU. For a program to be executed, it should be first loaded in the main memory. An Operating System performs the following activities for memory management:

It keeps track of primary memory, i.e., which bytes of memory are used by which user program. The memory addresses that have already been allocated and the memory addresses of the memory that has not yet been used. In multiprogramming, the OS decides the order in which processes are granted access to memory, and for how long. It Allocates the memory to a process when the process requests it and deallocates the memory when the process has terminated or is performing an I/O operation.

7. **Processor Management –**
   In a multi-programming environment, the OS decides the order in which processes have access to the processor, and how much processing time each process has. This function of OS is called process scheduling. An Operating System performs the following activities for processor management.

Keeps track of the status of processes. The program which performs this task is known as a traffic controller. Allocates the CPU that is a processor to a process. De-allocates processor when a process is no more required.

8. **Device Management –**
   An OS manages device communication via their respective drivers. It performs the following activities for device management. Keeps track of all devices connected to the system. designates a program responsible for every device known as the Input/Output controller. Decides which process gets access to a certain device and for how long. Allocates devices in an effective and efficient way. Deallocates devices when they are no longer required.

9. **File Management –**
   A file system is organized into directories for efficient or easy navigation and usage. These directories may contain other directories and other files. An Operating System carries out the following file management activities. It keeps track of where information is stored, user access settings and status of every file, and more… These facilities are collectively known as the file system.

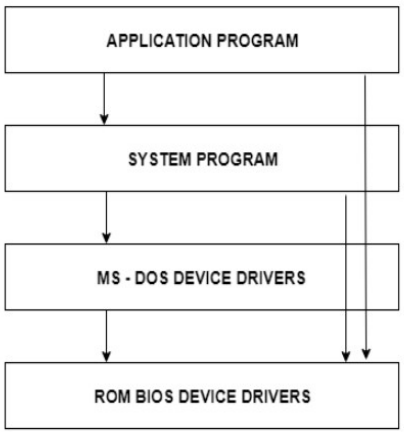
4 Operating System Structure

An operating system is a construct that allows the user application programs to interact with the system hardware. Since the operating system is such a complex structure, it should be created with utmost care so it can be used and modified easily. An easy way to do this is to create the operating system in parts. Each of these parts should be well defined with clear inputs, outputs and functions.

4.1 Simple Structure

There are many operating systems that have a rather simple structure. These started as small systems and rapidly expanded much further than their scope. A common example of this is MS-DOS. It was designed simply for a niche amount for people. There was no indication that it would become so popular.

An image to illustrate the structure of MS-DOS is as follows −



MS-DOS STRUCTURE

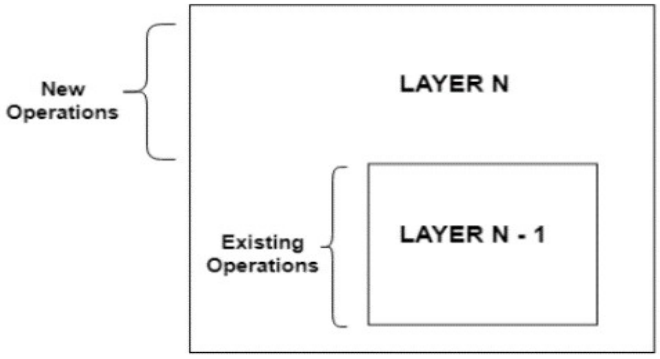
It is better that operating systems have a modular structure, unlike MS-DOS. That would lead to greater control over the computer system and its various applications. The modular structure would also

allow the programmers to hide information as required and implement internal routines as they see fit without changing the outer specifications.

4.2 Layered Structure

One way to achieve modularity in the operating system is the layered approach. In this, the bottom layer is the hardware and the topmost layer is the user interface.

An image demonstrating the layered approach is as follows −



Layered Structure of Operating System

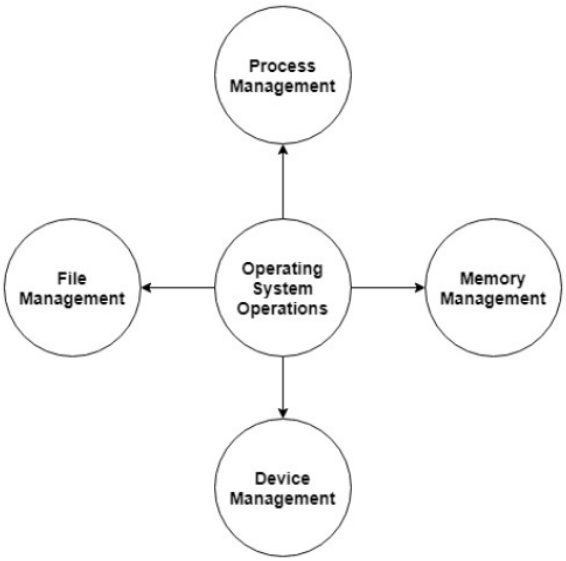
As seen from the image, each upper layer is built on the bottom layer. All the layers hide some structures, operations etc from their upper layers.

One problem with the layered structure is that each layer needs to be carefully defined. This is necessary because the upper layers can only use the functionalities of the layers below them.

5 Operating System Operations

An operating system is a construct that allows the user application programs to interact with the system hardware. Operating system by itself does not provide any function but it provides an atmosphere in which different applications and programs can do useful work.

The major operations of the operating system are process management, memory management, device management and file management. These are given in detail as follows:



5.1 Process Management

The operating system is responsible for managing the processes i.e assigning the processor to a process at a time. This is known as process scheduling. The different algorithms used for process scheduling are FCFS (first come first served), SJF (shortest job first), priority scheduling, round robin scheduling etc.

There are many scheduling queues that are used to handle processes in process management. When the processes enter the system, they are put into the job queue. The processes that are ready to execute in

the main memory are kept in the ready queue. The processes that are waiting for the I/O device are kept in the device queue.

5.2 Memory Management

Memory management plays an important part in operating system. It deals with memory and the moving of processes from disk to primary memory for execution and back again.

The activities performed by the operating system for memory management are −

- The operating system assigns memory to the processes as required. This can be done using best fit, first fit and worst fit algorithms.
- All the memory is tracked by the operating system i.e. it nodes what memory parts are in use by the processes and which are empty.
- The operating system delocated memory from processes as required. This may happen when a process has been terminated or if it no longer needs the memory.

5.3 Device Management

There are many I/O devices handled by the operating system such as mouse, keyboard, disk drive etc. There are different device drivers that can be connected to the operating system to handle a specific device. The device controller is an interface between the device and the device driver. The user applications can access all the I/O devices using the device drivers, which are device specific codes.

5.4 File Management

Files are used to provide a uniform view of data storage by the operating system. All the files are mapped onto physical devices that are usually non volatile so data is safe in the case of system failure.

The files can be accessed by the system in two ways i.e. sequential access and direct access −

- **Sequential Access**

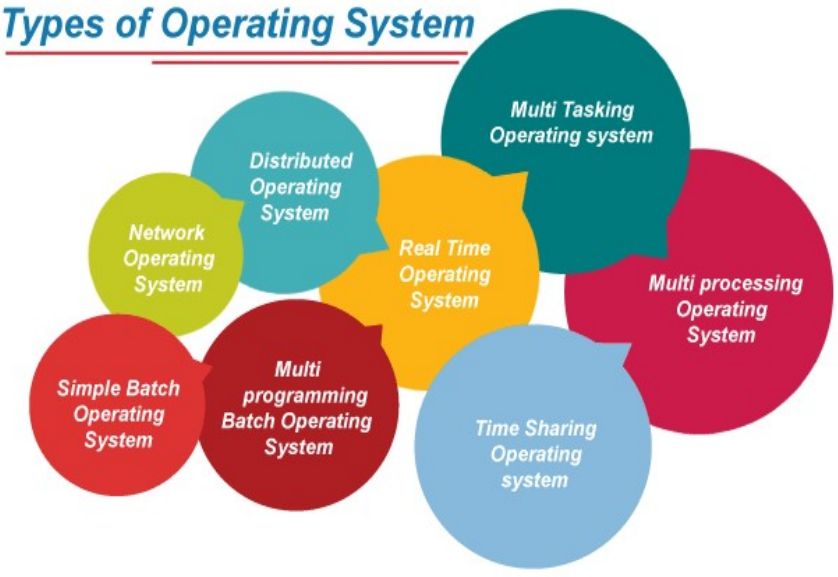
  The information in a file is processed in order using sequential access. The files records are accessed on after another. Most of the file systems such as editors, compilers etc. use sequential access.

- **Direct Access**

  In direct access or relative access, the files can be accessed in random for read and write operations. The direct access model is based on the disk model of a file, since it allows random accesses.

6 Types of Operating Systems

An operating system is a well-organized collection of programs that manages the computer hardware. It is a type of system software that is responsible for the smooth functioning of the computer system.

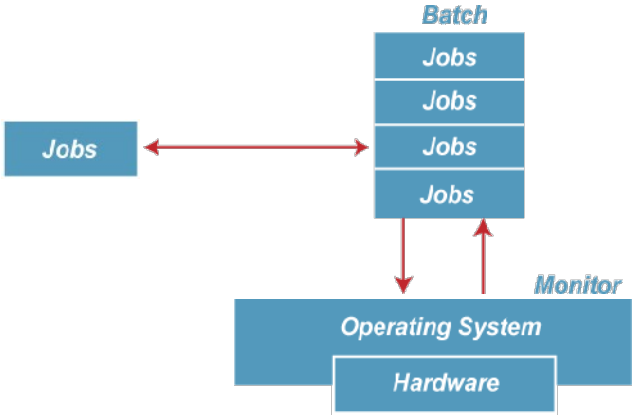


6.1 Batch Operating System

In the 1970s, Batch processing was very popular. In this technique, similar types of jobs were batched together and executed in time. People were used to having a single computer which was called a mainframe.

In Batch operating system, access is given to more than one person; they submit their respective jobs to the system for the execution.

The system put all of the jobs in a queue on the basis of first come first serve and then executes the jobs one by one. The users collect their respective output when all the jobs get executed.



The purpose of this operating system was mainly to transfer control from one job to another as soon as the job was completed. It contained a small set of programs called the resident monitor that always resided in one part of the main memory. The remaining part is used for servicing jobs.

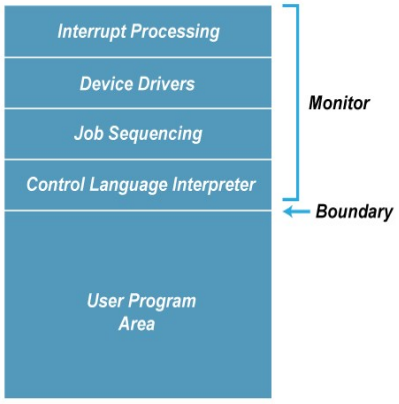


Figure: Memory Layout of the resident monitor

Advantages of Batch OS

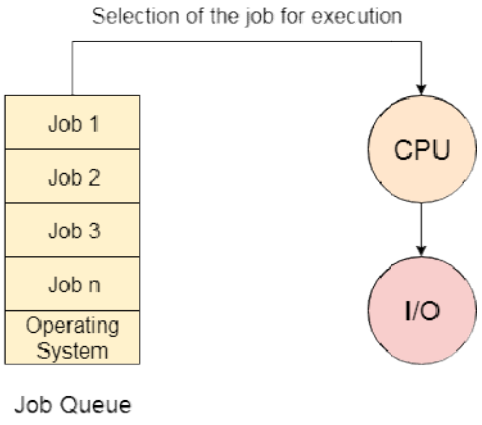
- The use of a resident monitor improves computer efficiency as it eliminates CPU time between two jobs.

Disadvantages of Batch OS

1. Starvation

Batch processing suffers from starvation.

For Example:



There are five jobs J1, J2, J3, J4, and J5, present in the batch. If the execution time of J1 is very high, then the other four jobs will never be executed, or they will have to wait for a very long time. Hence the other processes get starved.

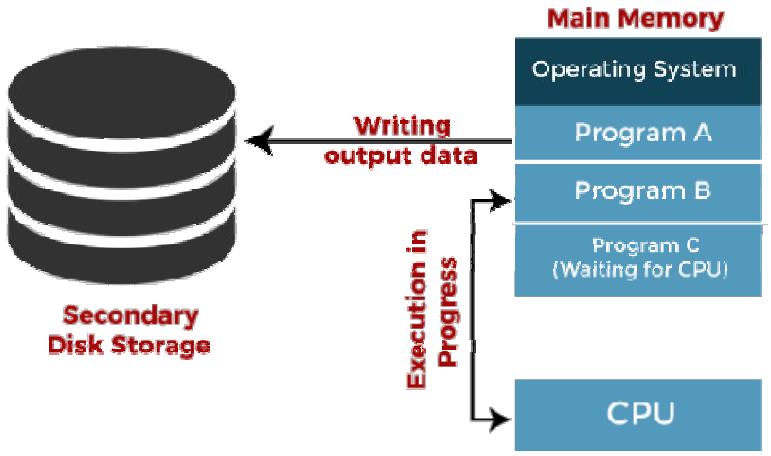
2. Not Interactive

Batch Processing is not suitable for jobs that are dependent on the user's input. If a job requires the input of two numbers from the console, then it will never get it in the batch processing scenario since the user is not present at the time of execution.

6.2 Multiprogramming Operating System

Multiprogramming is an extension to batch processing where the CPU is always kept busy. Each process needs two types of system time: CPU time and IO time.

In a multiprogramming environment, when a process does its I/O, The CPU can start the execution of other processes. Therefore, multiprogramming improves the efficiency of the system.



Jobs in multiprogramming system

Advantages of Multiprogramming OS

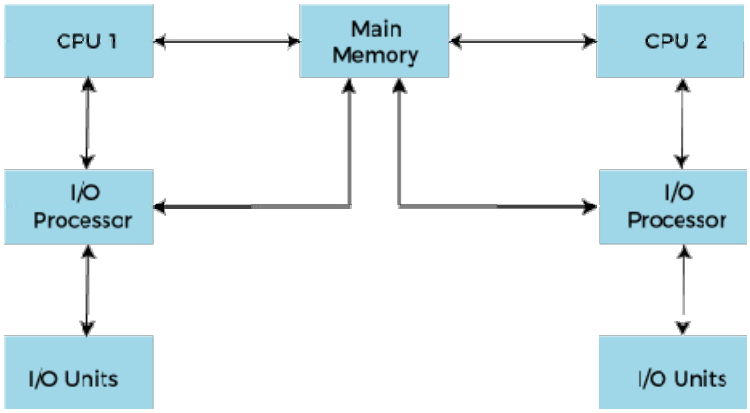
- Throughout the system, it increased as the CPU always had one program to execute.
- Response time can also be reduced.

Disadvantages of Multiprogramming OS

- Multiprogramming systems provide an environment in which various systems resources are used efficiently, but they do not provide any user interaction with the computer system.

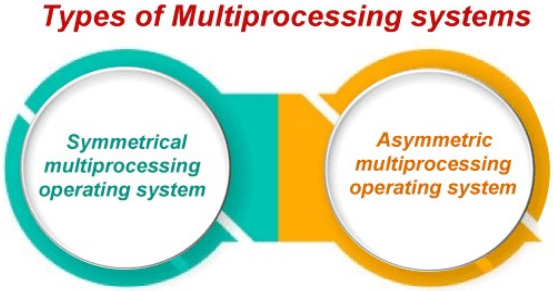
6.3 Multiprocessing Operating System

In Multiprocessing, Parallel computing is achieved. There are more than one processors present in the system which can execute more than one process at the same time. This will increase the throughput of the system.



Working of Multiprocessor System

In Multiprocessing, Parallel computing is achieved. More than one processor present in the system can execute more than one process simultaneously, which will increase the throughput of the system.



Advantages of Multiprocessing operating system:

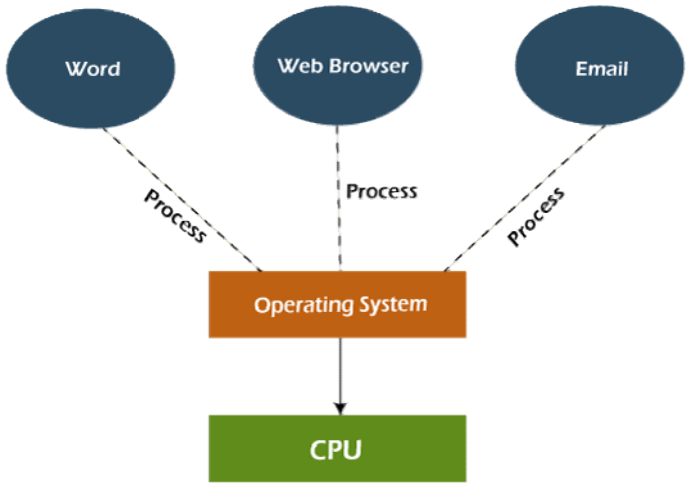
- **Increased reliability:** Due to the multiprocessing system, processing tasks can be distributed among several processors. This increases reliability as if one processor fails, the task can be given to another processor for completion.
- **Increased throughout:** As several processors increase, more work can be done in less.

Disadvantages of Multiprocessing operating System

- Multiprocessing operating system is more complex and sophisticated as it takes care of multiple CPUs simultaneously.

6.4 Multitasking Operating System

The multitasking operating system is a logical extension of a multiprogramming system that enables **multiple** programs simultaneously. It allows a user to perform more than one computer task at the same time.



Advantages of Multitasking operating system

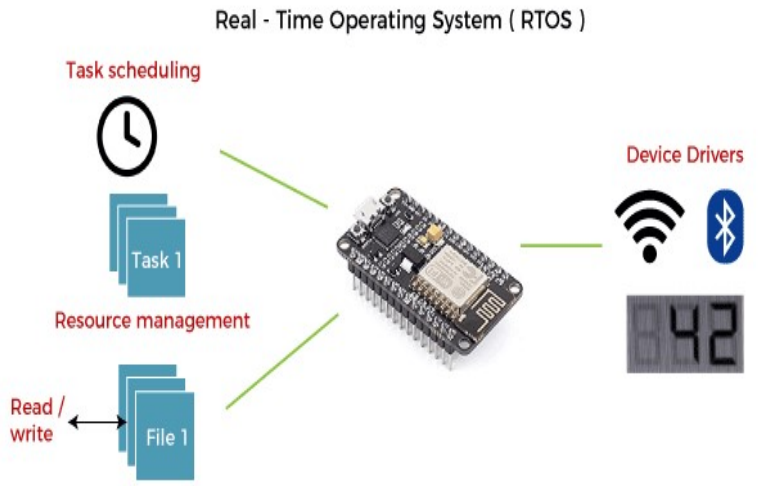
- This operating system is more suited to supporting multiple users simultaneously.
- The multitasking operating systems have well-defined memory management.

Disadvantages of Multitasking operating system

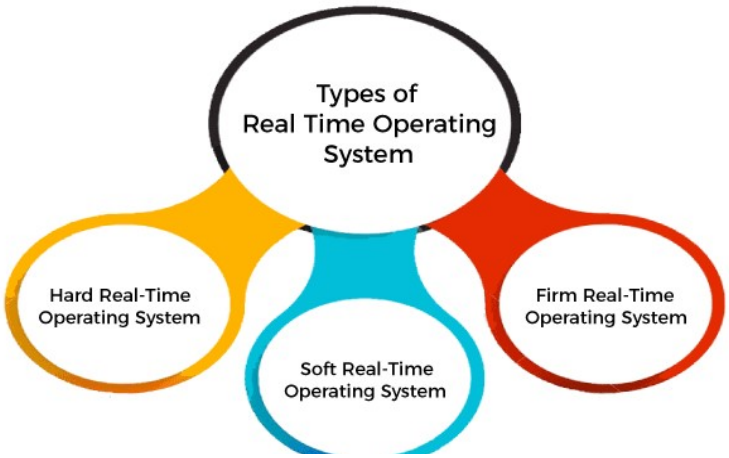
- The multiple processors are busier at the same time to complete any task in a multitasking environment, so the CPU generates more heat.

6.5 Real Time Operating System

In Real-Time Systems, each job carries a certain deadline within which the job is supposed to be completed, otherwise, the huge loss will be there, or even if the result is produced, it will be completely useless.



The Application of a Real-Time system exists in the case of military applications, if you want to drop a missile, then the missile is supposed to be dropped with a certain precision.

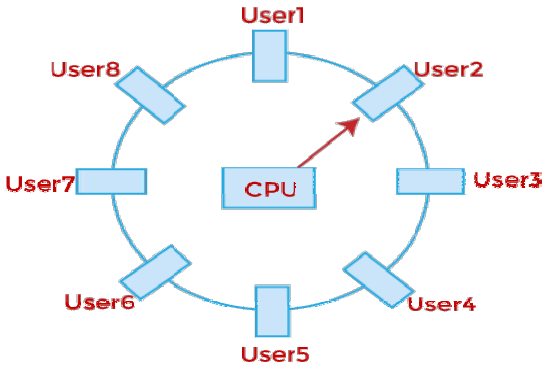


Advantages of Real-time operating system:

- Easy to layout, develop and execute real-time applications under the real-time operating system.
- In a Real-time operating system, the maximum utilization of devices and systems.

6.6 Time-Sharing Operating System

In the Time Sharing operating system, computer resources are allocated in a time-dependent fashion to several programs simultaneously. Thus it helps to provide a large number of user's direct access to the main computer. It is a logical extension of multiprogramming. In time-sharing, the CPU is switched among multiple programs given by different users on a scheduled basis.



A time-sharing operating system allows many users to be served simultaneously, so sophisticated CPU scheduling schemes and Input/output management are required.

Time-sharing operating systems are very difficult and expensive to build.

Advantages of Time Sharing Operating System

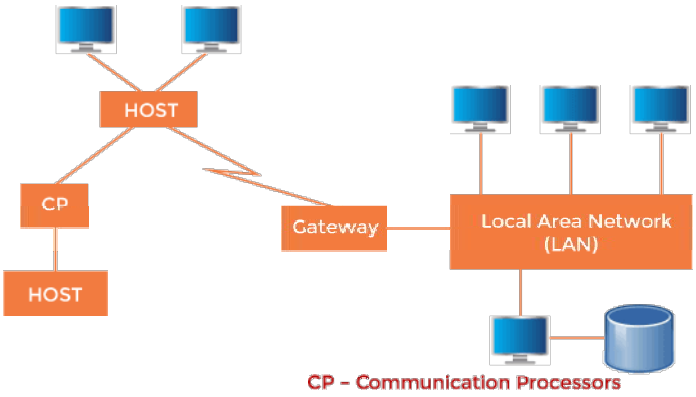
- The time-sharing operating system provides effective utilization and sharing of resources.
- This system reduces CPU idle and response time.

Disadvantages of Time Sharing Operating System

- Data transmission rates are very high in comparison to other methods.
- Security and integrity of user programs loaded in memory and data need to be maintained as many users access the system at the same time.

6.7 Distributed Operating System

The Distributed Operating system is not installed on a single machine, it is divided into parts, and these parts are loaded on different machines. A part of the distributed Operating system is installed on each machine to make their communication possible. Distributed Operating systems are much more complex, large, and sophisticated than Network operating systems because they also have to take care of varying networking protocols.



A Typical View of a Distributed System

Advantages of Distributed Operating System

- The distributed operating system provides sharing of resources.
- This type of system is fault-tolerant.

Disadvantages of Distributed Operating System

- Protocol overhead can dominate computation cost

7 Operating System Services-

Moreover, Operating System also provides certain services to the computer system in one form or the other.
The Operating System provides certain services to the users which can be listed in the following manner:

- **Program Execution:** The Operating System is responsible for the execution of all types of programs whether it be user programs or system programs. The Operating System utilizes various resources available for the efficient running of all types of functionalities.

- **Handling Input/output Operations**: The Operating System is responsible for handling all sorts of inputs, i.e, from the keyboard, mouse, desktop, etc. The Operating System does all interfacing in the most appropriate manner regarding all kinds of Inputs and Outputs.
  For example, there is a difference in the nature of all types of peripheral devices such as mice or keyboards; the Operating System is responsible for handling data between them.

- **Manipulation of File System:** The Operating System is responsible for making decisions regarding the storage of all types of data or files, i.e., floppy disk/hard disk/pen drive, etc. The Operating System decides how the data should be manipulated and stored.

- **Error Detection and Handling**: The Operating System is responsible for the detection of any type of error or bugs that can occur while any task. The well-secured OS sometimes also

acts as a countermeasure for preventing any sort of breach to the Computer System from any external source and probably handling them.

- **Resource Allocation:** The Operating System ensures the proper use of all the resources available by deciding which resource to be used by whom for how much time. All the decisions are taken by the Operating System.

- **Accounting:** The Operating System tracks an account of all the functionalities taking place in the computer system at a time. All the details such as the types of errors that occurred are recorded by the Operating System.

- **Information and Resource Protection:** The Operating System is responsible for using all the information and resources available on the machine in the most protected way. The Operating System must foil an attempt from any external resource to hamper any sort of data or information.

- All these services are ensured by the Operating System for the convenience of the users to make the programming task easier. All different kinds of Operating systems more or less provide the same services.

8 System Call-

A system call is a method for a computer program to request a service from the kernel of the operating system on which it is running. A system call is a method of interacting with the operating system via programs. A system call is a request from computer software to an operating system's kernel.

The **Application Program Interface (API)** connects the operating system's functions to user programs. It acts as a link between the operating system and a process, allowing user-level programs to request operating system services. The kernel system can only be accessed using system calls. System calls are required for any programs that use resources and user programs.

Below are some examples of how a system call varies from a user function.

1. A system call function may create and use kernel processes to execute the asynchronous processing.
2. A system call has greater authority than a standard subroutine. A system call with kernel-mode privilege executes in the kernel protection domain.
3. System calls are not permitted to use shared libraries or any symbols that are not present in the kernel protection domain.
4. The code and data for system calls are stored in global kernel memory.

Why do you need system calls in the Operating System?

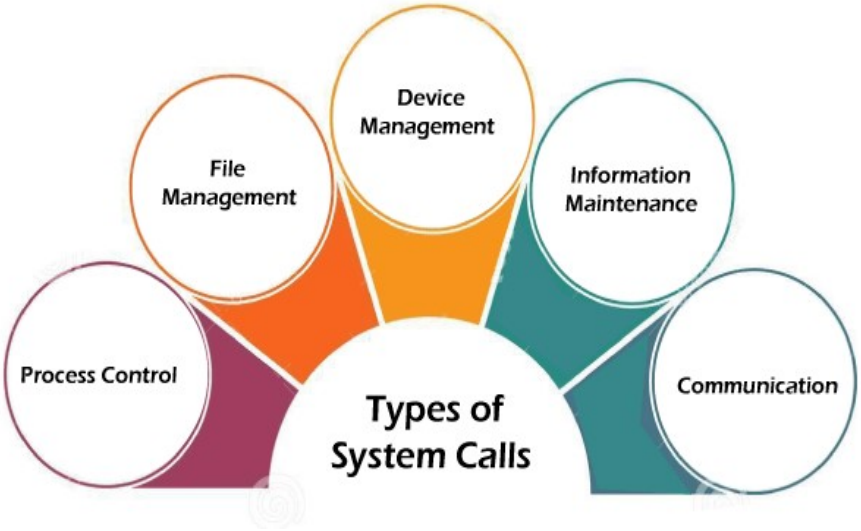
There are various situations where you must require system calls in the operating system. Following of the situations are as follows:

1. It is required when a file system wants to create or delete a file.
2. Network connections require the system to send and receiving data packets.
3. If you want to read or write a file, you need to system calls.
4. If you want to access hardware devices, including a printer, scanner, you need a system call.

5. System calls are used to create and manage new processes.

9 Types of System Calls

There are commonly five types of system calls. These are as follows:



- **Process Control**

  Process control is the system call that is used to direct the processes. Some process control examples include creating, load, abort, and end, execute, process, terminate the process, etc.

- **File Management**

  File management is a system call that is used to handle the files. Some file management examples include creating files, delete files, open, close, read, write, etc.

- **Device Management**

  Device management is a system call that is used to deal with devices. Some examples of device management include read, device, write, get device attributes, release device, etc.

- **Information Maintenance**

  Information maintenance is a system call that is used to maintain information. There are some examples of information maintenance, including getting system data, set time or date, get time or date, set system data, etc.

- **Communication**

  Communication is a system call that is used for communication. There are some examples of communication, including create, delete communication connections, send, receive messages, etc.

10 Examples of Windows and UNIX system calls

There are various examples of Windows and UNIX system calls. These are as listed below in the table:

Process	Windows	UNIX
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	Fork() Exit() Wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	Open() Read() Write() Close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	Ioctl() Read() Write()

Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	Getpid() Alarm() Sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	Pipe() Shmget() Mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorgroup()	Chmod() Umask() Chown()

----------END OF THE UNIT----------

Unit related questions

Q1. Define: i) Operating System, ii) System Call

Q2. Explain Operating System Structure including simple and layered structure.

Q3. Explain operating system operation in detail including file management, memory management, process management and device management.

Q4. What do you mean by Multi processing Operating System? Write its advantage and disadvantages.

Q5. What do you mean by Batch Operating System? Write its advantage and disadvantages.

Q6. What do you mean by Time sharing Operating System? Write its advantage and disadvantages.

Q7. What do you mean by Multi programming Operating System? Write its advantage and disadvantages.

Q8. What do you mean by Multi tasking Operating System? Write its advantage and disadvantages.

Q9. What do you mean by Real time Operating System? Write its advantage and disadvantages.

Q10. What do you mean by Distributed Operating System? Write its advantage and disadvantages.

Q11. Write about operating system services.

Q12. Explain types of system call in detail.

RENAISSANCE UNIVERSITY, INDORE
SCHOOL OF COMPUTER SCIENCE
Subject name: Operating System

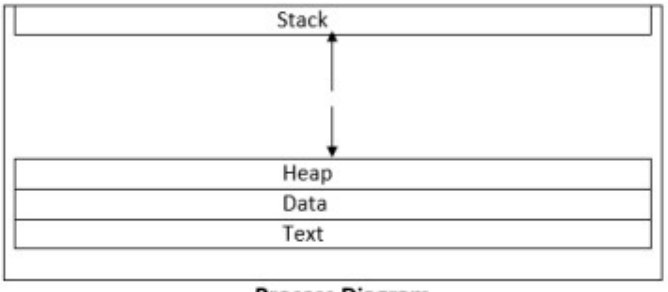
UNIT -2
Index

1. Process

It is an active program which running now on the Operating System is known as the process. The Process is the base of all computing things. Although process is relatively similar to the computer code but, the method is not the same as computer code. A process is an "active" entity, in contrast to the program, which is sometimes thought of as some sort of "passive" entity. The properties that the process holds include the state of the hardware, the RAM, the CPU, and other attributes.

2. Process in an Operating System

A process is actively running software or a computer code. Any procedure must be carried out in a precise order. An entity that helps in describing the fundamental work unit that must be implemented in any system is referred to as a process. In other words, we create computer programs as text files that, when executed, create processes that carry out all of the tasks listed in the program. When a program is loaded into memory, it may be divided into the four components stack, heap, text, and data to form a process. The simplified depiction of a process in the main memory is shown in the diagram below.



Process Diagram

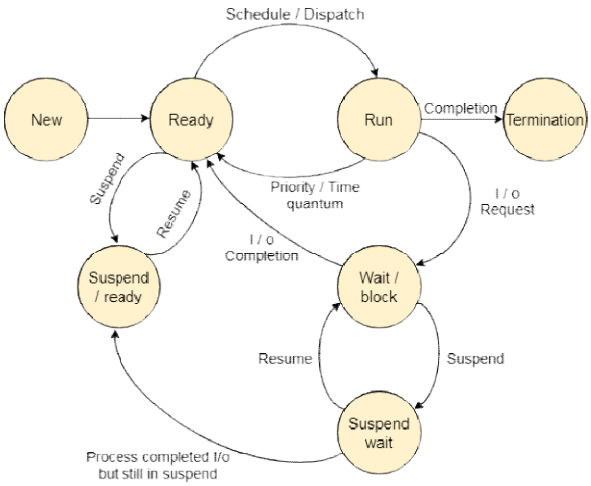
- **Stack:** The process stack stores temporary information such as method or function arguments, the return address, and local variables.
- **Heap**: This is the memory where a process is dynamically allotted while it is running.
- **Text**: This consists of the information stored in the processor's registers as well as the most recent activity indicated by the program counter's value.
- **Data:** In this section, both global and static variables are discussed.
- **Program:** It is a set of instructions which are executed when the certain task is allowed to complete that certain task. The programs are usually written in a Programming Language like C, C ++, Python, Java, R, C # (C sharp), etc. A computer program is a set of instructions that, when carried out by a computer, accomplish a certain task.

3. Difference between process and the program

S. No	Process	Program
1	A process is actively running software or a computer code. Any procedure must be carried out in a precise order. An entity that helps in describing the fundamental work unit that must be implemented in any system is referred to as a process	Program is a set of instructions which are executed when the certain task is allowed to complete that certain task
2	Process is Dynamic in Nature	Program is Static in Nature
3	Process is an Active in Nature	Program is Passive in Nature
4	Process is created during the execution and it is loaded directly into the main memory	Program is already existed in the memory and it is present in the secondary memory.
5	Process has its own control system known as Process Control Block	Program does not have any control system. It is just called when specified and it executes the whole program when called

6	Process changes from time to time by itself	Program cannot be changed on its own. It must be changed by the programmer.
7	A process needs extra data in addition to the program at a needed for management and execution.	Program is basically divided into two parts. One is Code part and the other part is data part.

4. Process State Diagram



The process, from its creation to completion, passes through various states. The minimum number of states is five.

The names of the states are not standardized although the process may be in one of the following states during execution.

1. New

A program which is going to be picked up by the OS into the main memory is called a new process.

2. Ready

Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned. The OS picks the new processes from the secondary memory and put all of them in the main memory.

The processes which are ready for the execution and reside in the main memory are called ready state processes. There can be many processes present in the ready state.

3. Running

One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm. Hence, if we have only one CPU in our system, the number of running processes for a particular time will always be one. If we have n processors in the system then we can have n processes running simultaneously.

4. Block or wait

From the Running state, a process can make the transition to the block or wait state depending upon the scheduling algorithm or the intrinsic behavior of the process.

When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.

5. Completion or termination

When a process finishes its execution, it comes in the termination state. All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.

6. Suspend ready

A process in the ready state, which is moved to secondary memory from the main memory due to lack of the resources (mainly primary memory) is called in the suspend ready state.

If the main memory is full and a higher priority process comes for the execution then the OS have to make the room for the process in the main memory by throwing the lower priority process out into the secondary memory. The suspend ready processes remain in the secondary memory until the main memory gets available.

7. Suspend wait

Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory. Since it is already waiting for some resource to get available hence it is better if it waits in the secondary memory and makes room for the higher priority process. These processes complete their execution once the main memory gets available and their wait is finished.

5. Various times related to process

1. Arrival time
2. Waiting time
3. Response time
4. Burst time
5. Completion time
6. Turn Around Time

1. Arrival Time-

Arrival time is the point of time at which a process enters the ready queue.

2. Waiting Time-

- Waiting time is the amount of time spent by a process waiting in the ready queue for getting the CPU.

Waiting time=Turnaround time –Burst time

3. Response time: Response time is the amount of time after which a process gets the CPU for the first time after entering the ready queue.

Response Time=Time at which process first gets the CPU–Arrival time

4. Burst Time-

- Burst time is the amount of time required by a process for executing on CPU.
- It is also called as **execution time** or **running time**.
- Burst time of a process can not be known in advance before executing the process.

It can be known only after the process has executed.

5. Completion Time-

- Completion time is the point of time at which process completes its execution on the CPU and takes exit from the system.
- It is also called as **exit time**.

6. Turnaround Time-

- Turn Around time is the total amount of time spent by a process in the system.
- When present in the system, a process is either waiting in the ready queue for getting the CPU or it is executing on the CPU.

Turnaround time=Burst time +Waiting time

OR

Turnaround time=Completion time– Arrival time

6. Operations on the Process

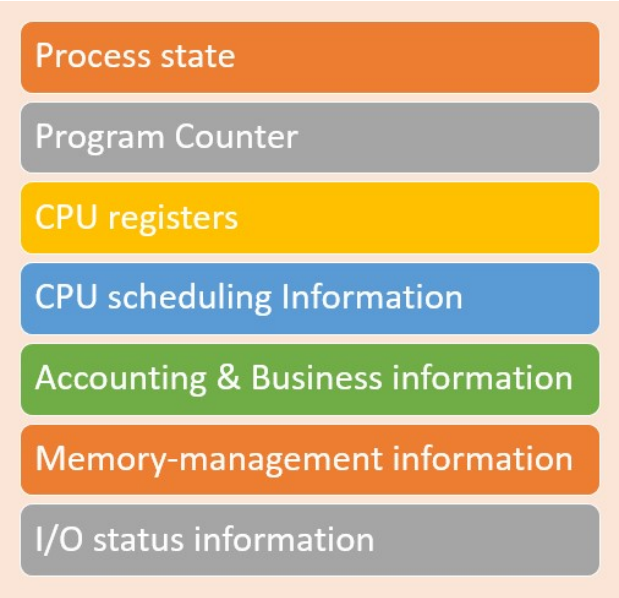
1. Creation: Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.

2. Scheduling: Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it. Selecting the process which is to be executed next, is known as scheduling.

3. Execution: Once the process is scheduled for the execution, the processor starts executing it. Process may come to the blocked or wait state during the execution then in that case the processor starts executing the other processes.

4. Deletion/killing: Once the purpose of the process gets over then the OS will kill the process. The Context of the process (PCB) will be deleted and the process gets terminated by the Operating system. **Process Control Block (PCB):** Every process is represented in the operating system by a process control block, which is also called a task control block.

Here, are important components of PCB are:



1. Process state: A process can be **new, ready, running, waiting**, etc.

2. Program counter: The program counter lets you know the address of the next instruction, which should be executed for that process.

3. CPU registers: This component includes accumulators, index and general-purpose registers, and information of condition code.

4. CPU scheduling information: This component includes a process priority, pointers for scheduling queues, and various other scheduling parameters.

5. Accounting and business information: It includes the amount of CPU and time utilities like real time used, job or process numbers, etc.

6. Memory-management information: This information includes the value of the base and limit registers, the page, or segment tables. This depends on the memory system, which is used by the operating system.

7. I/O status information: This block includes a list of open files, the list of I/O devices that are allocated to the process, etc.

7. Thread in Operating System

A thread is a single sequence stream within a process. Threads are also called lightweight processes as they possess some of the properties of processes. Each thread belongs to exactly one process. In an operating system that supports multithreading, the process can consist of many threads. But threads can be effective only if the CPU is more than 1 otherwise two threads have to context switch for that single CPU.

7.1 Components of Threads

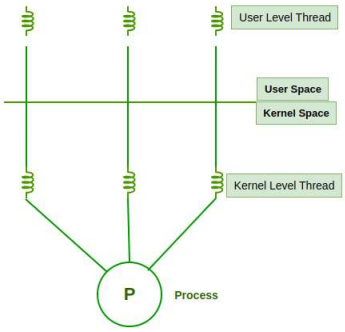
These are the basic components of the Operating System.

- Stack Space
- Register Set
- Program Counter

7.2 Types of Thread in Operating System

Threads are of two types. These are described below.

- User Level Thread
- Kernel Level Thread



7.2.1 User Level Threads

User Level Thread is a type of thread that is not created using system calls. The kernel has no work in the management of user-level threads. User-level threads can be easily implemented by the user. In case when user-level threads are single-handed processes, kernel-level thread manages them. Let's look at the advantages and disadvantages of User-Level Thread.

Advantages of User-Level Threads

- Implementation of the User-Level Thread is easier than Kernel Level Thread.
- Context Switch Time is less in User Level Thread.
- User-Level Thread is more efficient than Kernel-Level Thread.
- Because of the presence of only Program Counter, Register Set, and Stack Space, it has a simple representation.

Disadvantages of User-Level Threads

- There is a lack of coordination between Thread and Kernel.
- In case of a page fault, the whole process can be blocked.

7.2.2 Kernel Level Threads

A kernel Level Thread is a type of thread that can recognize the Operating system easily. Kernel Level Threads has its own thread table where it keeps track of the system. The operating System Kernel helps in managing threads. Kernel Threads have somehow longer context switching time. Kernel helps in the management of threads.

Advantages of Kernel-Level Threads

- It has up-to-date information on all threads.
- Applications that block frequency are to be handled by the Kernel-Level Threads.
- Whenever any process requires more time to process, Kernel-Level Thread provides more time to it.

Disadvantages of Kernel-Level threads

- Kernel-Level Thread is slower than User-Level Thread.
- Implementation of this type of thread is a little more complex than a user-level thread.

8. Types of Scheduler

1. Long term scheduler: Long term scheduler is also known as job scheduler. It chooses the processes from the pool (secondary memory) and keeps them in the ready queue maintained in the primary memory.

It mainly controls the degree of Multiprogramming. The purpose of long term scheduler is to choose a perfect mix of IO bound and CPU bound processes among the jobs present in the pool. If the job scheduler chooses more IO bound processes then all of the jobs may reside in the blocked state all the time and the CPU will remain idle most of the time. This will reduce the degree of Multiprogramming. Therefore, the Job of long term scheduler is very critical and may affect the system for a very long time.

2. Short term scheduler: Short term scheduler is also known as CPU scheduler. It selects one of the Jobs from the ready queue and dispatch to the CPU for the execution.

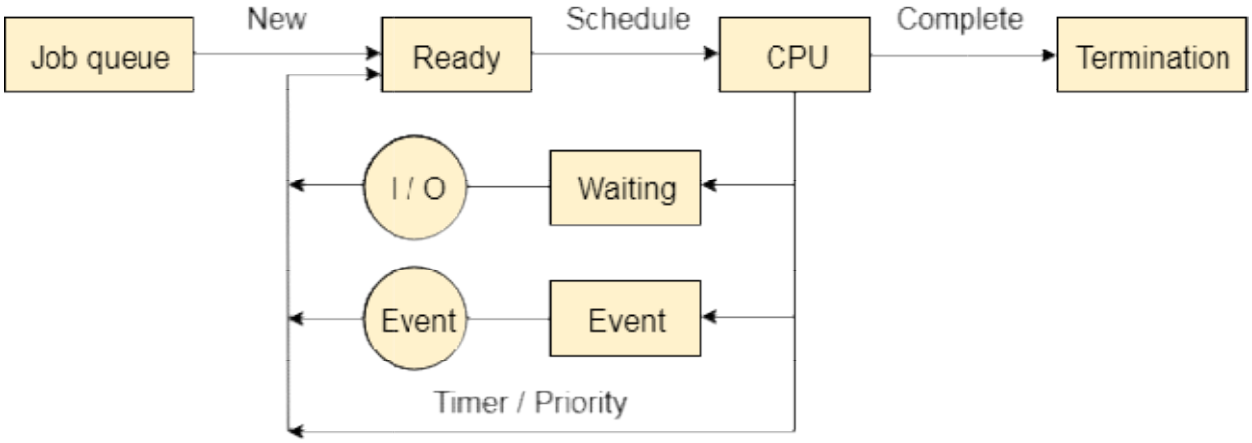
A scheduling algorithm is used to select which job is going to be dispatched for the execution. The Job of the short term scheduler can be very critical in the sense that if it selects job whose CPU burst time is very high then all the jobs after that, will have to wait in the ready queue for a very long time. This problem is called starvation which may arise if the short term scheduler makes some mistakes while selecting the job.

3. Medium term scheduler: Medium term scheduler takes care of the swapped out processes. If the running state processes needs some IO time for the completion then there is a need to change its state from running to waiting. Medium term scheduler is used for this purpose. It removes the process from the running state to make room for the other processes.

Such processes are the swapped out processes and this procedure is called swapping. The medium term scheduler is responsible for suspending and resuming the processes. It reduces the degree of multiprogramming. The swapping is necessary to have a perfect mix of processes in the ready queue.

9. Process Queues

The Operating system manages various types of queues for each of the process states. The PCB related to the process is also stored in the queue of the same state. If the Process is moved from one state to another state then its PCB is also unlinked from the corresponding queue and added to the other state queue in which the transition is made.



There are the following queues maintained by the Operating system.

1. Job Queue

In starting, all the processes get stored in the job queue. It is maintained in the secondary memory. The long term scheduler (Job scheduler) picks some of the jobs and put them in the primary memory.

2. Ready Queue

Ready queue is maintained in primary memory. The short term scheduler picks the job from the ready queue and dispatch to the CPU for the execution.

3. Waiting Queue

When the process needs some IO operation in order to complete its execution, OS changes the state of the process from running to waiting. The context (PCB) associated with the process gets stored on the waiting queue which will be used by the Processor when the process finishes the IO.

10. Context Switching

Context switching in an operating system involves saving the context or state of a running process so that it can be restored later, and then loading the context or state of another. Process and run it.

Context Switching refers to the process/method used by the system to change the process from one state to another using the CPUs present in the system to perform its job.

Example of Context Switching

Suppose in the OS there (N) numbers of processes are stored in a Process Control Block (PCB). Like The process is running using the CPU to do its job. While a process is running, other processes with the highest priority queue up to use the CPU to complete their job.
Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process. This task is known as a context switch. When a context switch occurs, the kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run. Context-switch time is pure overhead because the system does no useful work while switching. Switching speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions (such as a single instruction to load or store all registers). A typical speed is a few milliseconds. Context-switch times are highly dependent on hardware support. For instance, some processors (such as the Sun UltraSPARC) provide multiple sets of registers. A context switch here simply requires changing the pointer to the current register set. Of course, if there are more active processes than there are register sets, the system resorts to copying register data to and from memory, as before. Also, the more complex the operating system, the greater the amount of work that must be done during a context switch/

10.1 Need of Context Switching

Context switching enables all processes to share a single CPU to finish their execution and store the status of the system's tasks. The execution of the process begins at the same place where there is a conflict when the process is reloaded into the system.
The operating system's need for context switching is explained by the reasons listed below.

- One process does not directly switch to another within the system. Context switching makes it easier for the operating system to use the CPU's resources to carry out its tasks and store its context while switching between multiple processes.
- Context switching enables all processes to share a single CPU to finish their execution and store the status of the system's tasks. The execution of the process begins at the same place where there is a conflict when the process is reloaded into the system.
- Context switching only allows a single CPU to handle multiple processes requests parallels without the need for any additional processors.

11. Preemptive Scheduling

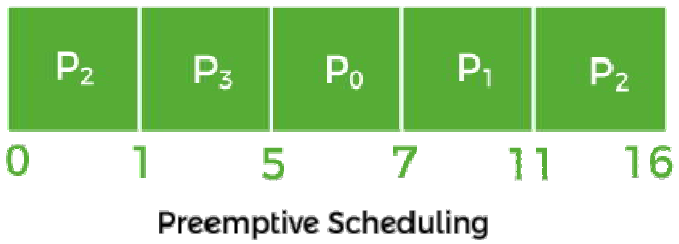
Preemptive scheduling is a method that may be used when a process switches from a running state to a ready state or from a waiting state to a ready state. The resources are assigned to the process for a particular time and then removed. If the resources still have the remaining CPU burst time, the process is placed back in the ready queue. The process remains in the ready queue until it is given a chance to execute again.

When a high-priority process comes in the ready queue, it doesn't have to wait for the running process to finish its burst time. However, the running process is interrupted in the middle of

its execution and placed in the ready queue until the high-priority process uses the resources. As a result, each process gets some CPU time in the ready queue. It improves the overhead of switching a process from running to ready state and vice versa, increasing preemptive scheduling flexibility. It may or may not include SJF and Priority scheduling.

For example: Let us take the example of Preemptive Scheduling. We have taken P0, P1, P2, and P3 are the four processes.

Process	Arrival Time	CPU Burst time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Preemptive Scheduling

- Firstly, the process P2 comes at time 0. So, the CPU is assigned to process P2.
- When process P2 was running, process P3 arrived at time 1, and the remaining time for process P2 (5 ms) is greater than the time needed by process P3 (4 ms). So, the processor is assigned to P3.
- When process P3 was running, process P1 came at time 2, and the remaining time for process P3 (3 ms) is less than the time needed by processes P1 (4 ms) and P2 (5 ms). As a result, P3 continues the execution.

- When process P3 continues the process, process P0 arrives at time 3. P3's remaining time (2 ms) is equal to P0's necessary time (2 ms). So, process P3 continues the execution.

- When process P3 finishes, the CPU is assigned to P0, which has a shorter burst time than the other processes.

- After process P0 completes, the CPU is assigned to process P1 and then to process P2.

11.1 Advantages and disadvantages of Preemptive Scheduling

Advantages
1. It is a more robust method because a process may not monopolize the processor.
2. Each event causes an interruption in the execution of ongoing tasks.
3. It improves the average response time.
4. It is more beneficial when you use this method in a multiprogramming environment.
5. The operating system ensures that all running processes use the same amount of CPU.

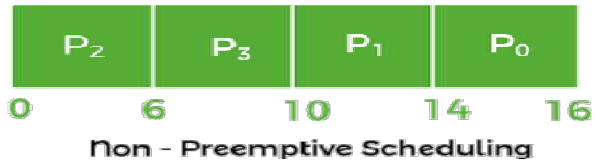
Disadvantages
1. It requires the use of limited computational resources.
2. It takes more time suspending the executing process, switching the context, and dispatching the new incoming process.
3. If several high-priority processes arrive at the same time, the lowpriority process would have to wait longer.

12. Non-Preemptive Scheduling

Non-preemptive scheduling is a method that may be used when a process terminates or switches from a running to a waiting state. When processors are assigned to a process, they keep the process until it is eliminated or reaches a waiting state. When the processor starts the process execution, it must complete it before executing the other process, and it may not be interrupted in the middle.

When a non-preemptive process with a high CPU burst time is running, the other process would have to wait for a long time, and that increases the process average waiting time in the ready queue. However, there is no overhead in transferring processes from the ready queue to the CPU under non-preemptive scheduling. The scheduling is strict because the execution process is not even preempted for a higher priority process.

For example: Let's take the above preemptive scheduling example and solve it in a non preemptive manner



- The process P2 comes at 0, so the processor is assigned to process P2 and takes (6 ms) to execute.
- All of the processes, P0, P1, and P3, arrive in the ready queue in between. But all processes wait till process P2 finishes its CPU burst time.
- After that, the process that comes after process P2, i.e., P3, is assigned to the CPU until it finishes its burst time.
- When process P1 completes its execution, the CPU is given to process P0.

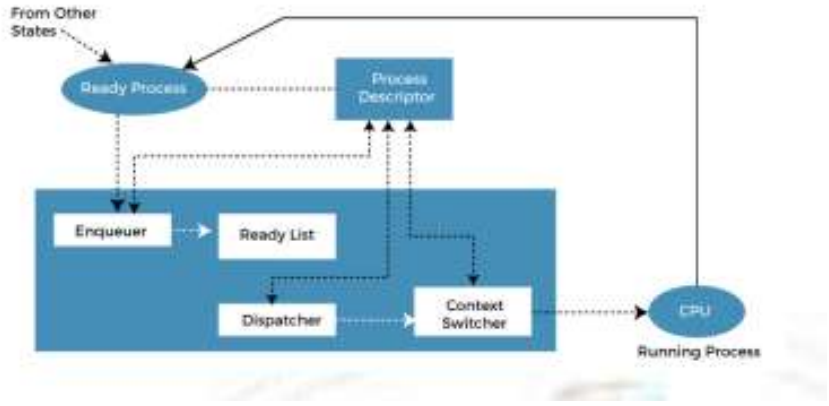
12.1 Advantages and disadvantages of Non-preemptive Scheduling:

Advantages
1. It provides a low scheduling overhead.
2. It is a very simple method.
3. It uses less computational resources.
4. It offers high throughput.

Disadvantages
1. It has a poor response time for the process.
2. A machine can freeze up due to bugs.

13. Dispatcher in OS

A dispatcher is a special program that comes into play after the scheduler. When the short term scheduler selects from the ready queue, the Dispatcher performs the task of allocating the selected process to the CPU. A running process goes to the waiting state for IO operation etc., and then the CPU is allocated to some other process. This switching of CPU from one process to the other is called context switching.



A dispatcher performs various tasks, including context switching, setting up user registers and memory mapping. These are necessary for the process to execute and transfer CPU control to that process. When dispatching, the process changes from the ready state to the running state.

The Dispatcher needs to be as fast as possible, as it is run on every context switch. The time consumed by the Dispatcher is known as dispatch latency. Sometimes, the Dispatcher is considered part of the short-term scheduler, so the whole unit is called the short-term

scheduler. In this scenario, the task of the short term scheduler is to select a process from the ready queue and allocate the CPU for that process.

In the operating system, a dispatcher has the following responsibilities:

1. **Switching to user mode:** All of the low-level operating system processes run on the kernel level security access, but all application code and user issued processes run in the application space or the user permission mode. The Dispatcher switches the processes to the user mode.

2. **Addressing:** The program counter (PC) register points towards the next process to be executed. The Dispatcher is responsible for addressing that address.

3. **Initiation of context switch:** A context switch is when a currently running process is halted, and all of its data and its process control block (PCB) are stored in the main memory, and another process is loaded in its place for execution.

4. **Managing dispatch latency:** Dispatch latency is calculated as the time it takes to stop one process and start another. The lower the dispatch latency, the more efficient the software for the same hardware configuration.

14. CPU Scheduling Criteria

- **CPU utilization**

The main objective of any CPU scheduling algorithm is to keep the CPU as busy as possible. Theoretically, CPU utilization can range from 0 to 100 but in a real-time system, it varies from 40 to 90 percent depending on the load upon the system.

- **Throughput**

A measure of the work done by the CPU is the number of processes being executed and completed per unit of time. This is called throughput. The throughput may vary depending on the length or duration of the processes.

- **Turnaround Time**

For a particular process, an important criterion is how long it takes to execute that process. The time elapsed from the time of submission of a process to the time of completion is known as the turnaround time. Turn-around time is the sum of times spent waiting to get into memory, waiting in the ready queue, executing in CPU, and waiting for I/O.

- **Waiting Time**

A scheduling algorithm does not affect the time required to complete the process once it starts execution. It only affects the waiting time of a process i.e. time spent by a process waiting in the ready queue.

- **Response Time**

In an interactive system, turn-around time is not the best criterion. A process may produce some output fairly early and continue computing new results while previous results are being output to the user. Thus another criterion is the time taken from submission of the process of the request until the first response is produced. This measure is called response time.

- **Completion Time**

The completion time is the time when the process stops executing, which means that the process has completed its burst time and is completely executed.

- **Priority**

If the operating system assigns priorities to processes, the scheduling mechanism should favor the higher-priority processes.

- **Predictability**

A given process always should run in about the same amount of time under a similar system load.

15. CPU Scheduling algorithms

There are various CPU Scheduling algorithms such as-

1. First Come First Served (FCFS)
2. Shortest Job First (SJF)
3. Longest Job First (LJF)
4. Priority Scheduling
5. Round Robin (RR)
6. Shortest Remaining Time First (SRTF)
7. Longest Remaining Time First (LRTF)

16. First Come First Serve

- In FCFS Scheduling, The process which arrives first in the ready queue is firstly assigned the CPU.
- In case of a tie, process with smaller process id is executed first.
- It is always non-preemptive in nature.

Advantages-

i. It is simple and easy to understand.

ii. It can be easily implemented using queue data structure.
iii. It does not lead to starvation.

Disadvantages-

i. It does not consider the priority or burst time of the processes.
ii. It suffers from convoy effect.

Convoy Effect: In convoy effect, Consider processes with higher burst time arrived before the processes with smaller burst time. Then, smaller processes have to wait for a long time for longer processes to release the CPU.

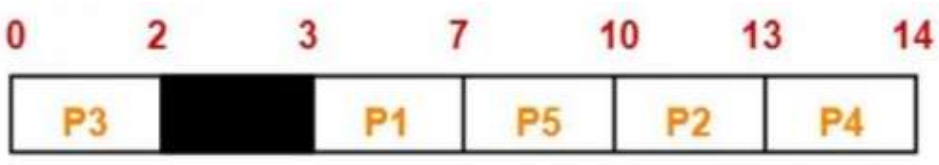
Problem-01: Consider the set of 5 processes whose arrival time and burst time are given below

Processes	Arrival Time	Burst Time
P1	3	4
P2	5	3
P3	0	2
P4	5	1
P5	1	3

If the CPU scheduling policy is FCFS,
calculate the average waiting time and average turnaround time.

Solution:

Gantt chart



Here, black box represents the idle time of CPU.

Now, we know-

Turn Around time = Exit time – Arrival time
Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	7	7 – 3 = 4	4 – 4 = 0
P2	13	13 – 5 = 8	8 – 3 = 5
P3	2	2 – 0 = 2	2 – 2 = 0
P4	14	14 – 5 = 9	9 – 1 = 8
P5	10	10 – 4 = 6	6 – 3 = 3

Now,
Average Turn Around time = (4 + 8 + 2 + 9 + 6) / 5 = 29 / 5 = 5.8 unit
Average waiting time = (0 + 5 + 0 + 8 + 3) / 5 = 16 / 5 = 3.2 unit

Problem-02: Consider the set of 3 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time
P1	0	2
P2	3	1
P3	5	6

If the CPU scheduling policy is FCFS, calculate the average waiting time and average turnaround time.

Solution
Gantt chart

Gantt Chart

Here, black box represents the idle time of CPU.

Now, we know-

Turn Around time = Exit time – Arrival time
Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	2	2 – 0 = 2	2 – 2 = 0
P2	4	4 – 3 = 1	1 – 1 = 0
P3	11	11- 5 = 6	6 – 6 = 0

Now,

Average Turn Around time = (2 + 1 + 6) / 3 = 9 / 3 = 3 unit
Average waiting time = (0 + 0 + 0) / 3 = 0 / 3 = 0 unit

17. SJF Scheduling (Shortest Job First)

In SJF Scheduling, Out of all the available processes, CPU is assigned to the process having smallest burst time.
In case of a tie, it is broken by FCFS Scheduling.
SJF Scheduling can be used in both preemptive and non-preemptive mode.

Preemptive mode of Shortest Job First is called as Shortest Remaining Time First (SRTF).

Advantages-

i. SRTF is optimal and guarantees the minimum average waiting time.
ii. It provides a standard for other algorithms since no other algorithm performs better than it.

Disadvantages-

i. It cannot be implemented practically since burst time of the processes can not be known in advance. It leads to starvation for processes with larger burst time.
ii. Priorities cannot be set for the processes.
iii. Processes with larger burst time have poor response time.

PRACTICE PROBLEMS BASED ON SJF SCHEDULING

Problem-01: Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

If the CPU scheduling policy is SJF non-preemptive, calculate the average waiting time and average turnaround time.

Solution

Gantt chart

Gantt Chart

Now, we know-

Turn Around time = Exit time – Arrival time
Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	7	7 – 3 = 4	4 – 1 = 3
P2	16	16 – 1 = 15	15 – 4 = 11
P3	9	9 – 4 = 5	5 – 2 = 3
P4	6	6 – 0 = 6	6 – 6 = 0
P5	12	12 – 2 = 10	10 – 3 = 7

Now,

- Average Turn Around time=(4+15+ 5+6+10)/5=40/5=8unit
- Average waiting time=(3+11+3+0+7)/5=24/5=4.8unit

Problem-02: Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	**Arrival Time**	**Burst Time**
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

If the CPU scheduling policy is SJF preemptive, calculate the average waiting time and average turnaround time.

Solution

Gantt chart

Gantt Chart

Now, we know-

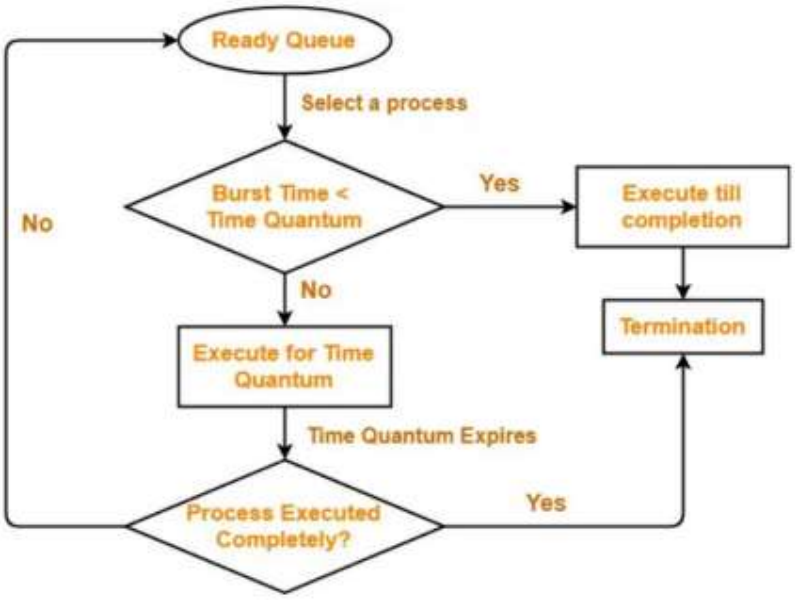
Turn Around time = Exit time – Arrival time
Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	4	4–3=1	1–1=0
P2	6	6–1=5	5–4=1
P3	8	8–4=4	4–2=2
P4	16	16–0=16	16–6=10
P5	11	11–2=9	9–3=6

Now,
Average Turn Around time = (1 + 5 + 4 + 16 + 9) / 5 = 35 / 5 = 7 unit
Average waiting time = (0 + 1 + 2 + 10 + 6) / 5 = 19 / 5 = 3.8 unit

18. Round Robin Scheduling

- In Round Robin Scheduling, CPU is assigned to the process on the basis of FCFS for a fixed amount of time.
- This fixed amount of time is called as time quantum or time slice.
- After the time quantum expires, the running process is preempted and sent to the ready queue. Then, the processor is assigned to the next arrived process.
- It is always preemptive in nature.
- Round Robin Scheduling is FCFS Scheduling with preemptive mode.



Round Robin Scheduling

Advantages-

i. It gives the best performance in terms of average response time
ii. It is best suited for time sharing system, client server architecture and interactive system.

Disadvantages-

i. It leads to starvation for processes with larger burst time as they have to repeat the cycle many times.
ii. Its performance heavily depends on time quantum.
iii. Priorities cannot be set for the processes.

Problem-01: Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Exit time	Turn Around time	Waiting time
P1	4	4–3=1	1–1=0

P2	6	6–1=5	5–4=1
P3	8	8–4=4	4–2=2
P4	16	16–0=16	16–6=10
P5	11	11–2=9	9–3=6

If the CPU scheduling policy is Round Robin with time quantum = 2 unit, calculate the average waiting time and average turnaround time.

Solution

Gantt chart

Ready Queue-

P1, P2, P3, P1, P4, P5, P2, P1,P5

Now, we know-

Turn Around time = Exit time – Arrival time
Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	13	13 – 0 = 13	13 – 5 = 8
P2	12	12 – 1 = 11	11 – 3 = 8
P3	5	5 – 2 = 3	3 – 1 = 2
P4	9	9 – 3 = 6	6 – 2 = 4
P5	14	14 – 4 = 10	10 – 3 = 7

Now,

Average Turn Around time = (13 + 11 + 3 + 6 + 10) / 5 = 43 / 5 = 8.6 unit

Average waiting time = (8 + 8 + 2 + 4 + 7) / 5 = 29 / 5 = 5.8 unit

19. Priority Scheduling

In Priority Scheduling, Out of all the available processes, CPU is assigned to the process having the highest priority.

In case of a tie, it is broken by FCFS Scheduling.

Priority Scheduling can be used in both preemptive and nonpreemptive mode.

Advantages-

It considers the priority of the processes and allows the important processes to run first.

Priority scheduling in preemptive mode is best suited for real time operating system.

Disadvantages-

Processes with lesser priority may starve for CPU.

There is no idea of response time and waiting time.

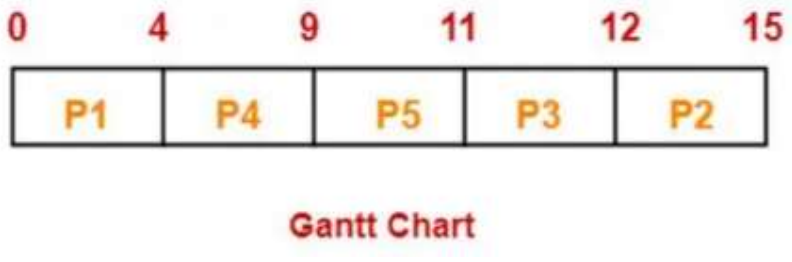
Problem-01: Consider the set of 5 processes whose arrival time and burst time are given below.

Processes Id	**Arrival time**	**Burst time**	**Priority**
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

If the CPU scheduling policy is priority non-preemptive, calculate the average waiting time and average turnaround time. (Higher number represents higher priority)

Solution

Gantt Chart-



Gantt Chart

Now,

Average Turn Around time = (13 + 11 + 3 + 6 + 10) / 5 = 43 / 5 = 8.6 unit

Average waiting time = (8 + 8 + 2 + 4 + 7) / 5 = 29 / 5 = 5.8 unit

20. Process Synchronization

It is the coordination of execution of multiple processes in a multi-process system to ensure that they access shared resources in a controlled and predictable manner. It aims to resolve the problem of race conditions and other synchronization issues in a concurrent system.
The main objective of process synchronization is to ensure that multiple processes access shared resources without interfering with each other and to prevent the possibility of inconsistent data due to concurrent access. To achieve this, various synchronization techniques such as semaphores, monitors, and critical sections are used.

21. Process

A process is a program that is currently running or a program under execution is called a process. It includes the program's code and all the activity it needs to perform its tasks, such as using the CPU, memory, and other resources. Think of a process as a task that the computer is working on, like opening a web browser or playing a video.

21.1Types of Process

On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process:** The execution of one process does not affect the execution of other processes.
- **Cooperative Process:** A process that can affect or be affected by other processes executing in the system.

22. Race Condition:

A race condition is a situation that may occur inside a critical section. This happens when the result of multiple thread execution in a critical section differs according to the order in which the threads execute. Race conditions in critical sections can be avoided if the critical section is treated as an atomic instruction.

23. Critical section:

It refers to a segment of code that is executed by multiple concurrent threads or processes, and which accesses shared resources. These resources may include shared memory, files, or other system resources that can only be accessed by one thread or process at a time to avoid data inconsistency or race conditions.

There are some properties that should be followed if any code in the critical section.

1. **Mutual Exclusion :** If process Pi is executing in its critical section, then no other processes can be executing in their critical sections.

2. **Progress :** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

3. **Bounded Waiting:** There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

24. Hardware synchronization: It is crucial in operating systems to ensure that multiple processes can access shared resources without conflicts. This is particularly important in a multiprocessor environment where processes may run concurrently and attempt to modify shared data simultaneously, leading to race conditions.

There are three algorithms in the hardware approach of solving Process Synchronization problem:
1. Test and Set
2. Swap
3. Unlock and Lock

Here's a simplified explanation of the three algorithms:

A. Test and Set

- A shared variable, lock, starts as false.
- **How it works:**
    - The TestAndSet function returns the current value of lock and sets it to true.
    - The first process sees lock = false, so it enters the critical section.
    - Other processes keep looping because lock = true.
    - When the first process finishes, it sets lock = false, letting another process enter.
- **Pros:** Mutual exclusion and progress are ensured.
- **Cons:** No guarantee of fairness (bounded waiting).

B. Swap

- Similar to TestAndSet, but uses an extra variable, key.
- **How it works:**
    - Each process sets key = true and swaps it with lock.
    - If key = false, the process enters the critical section.
    - While one process is in the critical section, lock = true, so others are blocked.
- **Pros:** Ensures mutual exclusion and progress.
- **Cons:** Like TestAndSet, it doesn't ensure fairness (bounded waiting).

C. Lock and Unlock

- Adds a waiting[i] array and uses a circular queue for fairness.
- **How it works:**
    - When a process finishes, it doesn't immediately set lock = false.
    - Instead, it checks if another process is waiting.
    - If a process is waiting, it gets priority and enters the critical section next.
    - If no one is waiting, lock = false, and any process can enter.
- **Pros:** Ensures mutual exclusion, progress, and fairness (bounded waiting).
- **Cons:** Slightly more complex than the other two.

25. Semaphores:

They are special variables used to manage processes in a computer system, ensuring they work smoothly without interfering with each other. They help with:

- **Mutual exclusion:** Ensuring only one process uses a resource at a time.
- **Synchronization:** Making processes wait for each other when needed.

25.1 How Semaphores Work:

1. **Two main operations:**
    - **Wait (P):** Decreases the semaphore value. If it becomes less than 0, the process waits (is blocked).

- **Signal (V):** Increases the semaphore value. If any process is waiting, one is allowed to continue.

2. **Example:**
    - A semaphore starts with a value (e.g., 3 means 3 processes can enter).
    - Each time a process uses the resource, wait decreases the value.
    - When the semaphore value is 0, no more processes can enter until a signal operation increases it again.

This ensures processes take turns, avoiding conflicts. Semaphores can also count how many resources are free, so they're called **counting semaphores**.

RENAISSANCE UNIVERSITY, INDORE
SCHOOL OF COMPUTER SCIENCE
Subject name: Operating System

UNIT RELATED QUESTIONS

Q.1 Define: i)Process, ii) Arrival ,iii) Waiting time ,iv) Response time, v) Burst time, vi) Turn Around Time ,vii) Completion time, viii)Kernel Level Thread, ix)User Level Thread, x)Critical Section, xi)Race Condition , xii) Mutual Exclusion, xiii) Process Synchronization.

Q.2 Write a short note on process in an operating system including process diagram.

Q.3 Write a difference between process and the program in tabular form.

Q.4 Explain process state diagram in detail including state diagram.

Q.5 Explain operation on the process in detail including diagram.

Q.6 Write a short note on thread including its types.

Q.7 What do you mean by process queue. Explain its types including diagram.

Q.8 Write a short note on types of scheduler.

Q.9 Write differences between preemptive scheduling and non preemptive scheduling in tabular form.

Q.10 Write a short note on dispatcher in operating system including diagram.

Q.11 Explain FCFS scheduling, including its advantages and disadvantages, and discuss any process-related problems that can be resolved using this scheduling techniques.

Q.12 Explain SJF scheduling, including its advantages and disadvantages, and discuss any process-related problems that can be resolved using this scheduling techniques.

Q.12 Explain Round Robin scheduling, including its advantages and disadvantages, and discuss any process-related problems that can be resolved using this scheduling techniques.

Q.12 Explain Priority scheduling, including its advantages and disadvantages, and discuss any process-related problems that can be resolved using this scheduling techniques..

Q.13 Mention CPU Scheduling criteria and define each of them.

Q. 14 Write a short note on semaphores.

Q.15 Write a short note on hardware synchronization.

Q.16 Write a short note on process synchronization.

Q.17 Write a short note on critical section and its necessary properties.

UNIT -3

Index

1. Memory Management

In a multiprogramming computer, the operating system resides in a part of memory and the rest is used by multiple processes. The task of subdividing the memory among different processes is called memory management. Memory management is a method in the operating system to manage operations between main memory and disk during process execution. The main aim of memory management is to achieve efficient utilization of memory.

2. Requirement of Memory Management

- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

3. Concept of logical address space and Physical address space

A. Logical Address space: An address generated by the CPU is known as "Logical Address". **It is also known as a Virtual address.** Logical address space can be defined as the size of the process. A logical address can be changed.

B. Physical Address space: An address seen by the memory unit (i.e. the one loaded into the memory address register of the memory) is commonly known as a **"Physical Address"**. A Physical address is also known as a Real address. The set of all physical addresses corresponding to these logical addresses is known as Physical address space. A physical address is computed by MMU. The run-time mapping from virtual to physical addresses is done by a hardware device Memory Management Unit (MMU). The physical address always remains constant.

4. Static and Dynamic Loading:

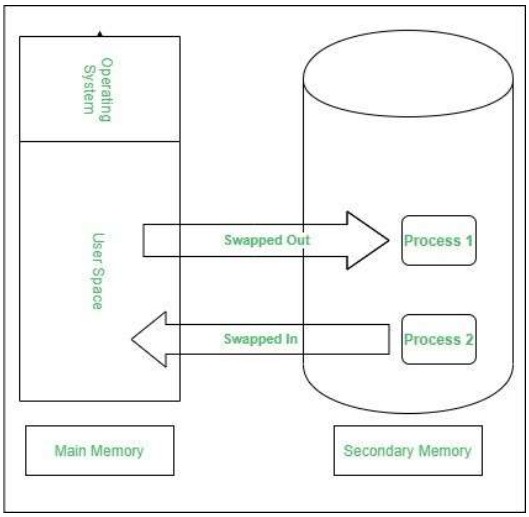
To load a process into the main memory is done by a loader. There are two different types of loading:

A. Static loading: loading the entire program into a fixed address. It requires more memory space.

B. Dynamic loading: The entire program and all data of a process must be in physical memory for the process to execute. So, the size of a process is limited to the size of physical memory. To gain proper memory utilization, dynamic loading is used. In dynamic loading, a routine is not loaded until it is called. All routines are residing on disk in a re-locatable load format. One of the advantages of dynamic loading is that unused routine is never loaded. This loading is useful when a large amount of code is needed to handle it efficiently.

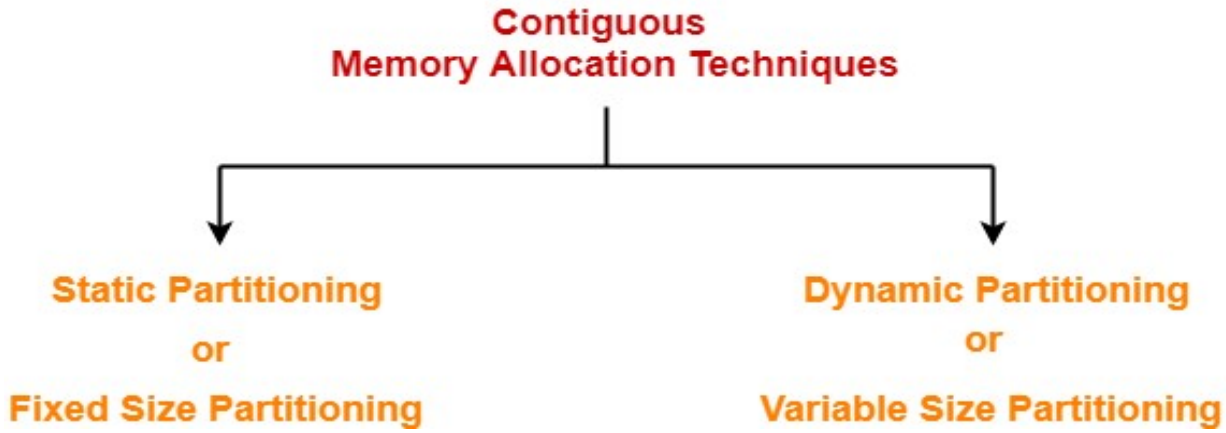
5. Swapping

When a process is executed it must have resided in memory. Swapping is a process of swap a process temporarily into a secondary memory from the main memory, which is fast as compared to secondary memory. A swapping allows more processes to be run and can be fit into memory at one time. The main part of swapping is transferred time and the total time directly proportional to the amount of memory swapped. Swapping is also known as roll-out, roll in, because if a higher priority process arrives and wants service, the memory manager can swap out the lower priority process and then load and execute the higher priority process. After finishing higher priority work, the lower priority process swapped back in memory and continued to the execution process.



6. Contiguous Memory Allocation

There are two popular techniques used for contiguous memory allocation-



7. Static Partitioning

- Static partitioning is a fixed size partitioning scheme.
- In this technique, main memory is pre-divided into fixed size partitions.
- The size of each partition is fixed and cannot be changed.
- Each partition is allowed to store only one process.

Example-

Under fixed size partitioning scheme, a memory of size 10 KB may be divided into fixed size partitions as-

5 KB	2 KB	2 KB	1 KB

- These partitions are allocated to the processes as they arrive.
- **The partition allocated to the arrived process depends on the algorithm followed.**

A. First Fit Algorithm-

- This algorithm starts scanning the partitions serially from the starting.
- When an empty partition that is big enough to store the process is found, it is allocated to the process.
- Obviously, the partition size has to be greater than or at least equal to the process size.

B. Best Fit Algorithm-

- This algorithm first scans all the empty partitions.
- It then allocates the smallest size partition to the process.

C. Worst Fit Algorithm-

- This algorithm first scans all the empty partitions.
- It then allocates the largest size partition to the process.

But because the space is not contiguous, so the process cannot be stored.

Translating Logical Address into Physical Address-

- CPU always generates a logical address.
- A physical address is needed to access the main memory.

Problem-01:

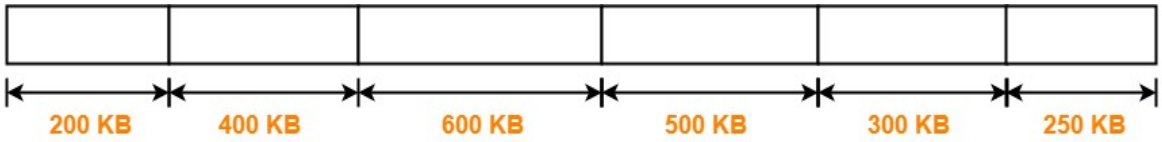
Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order.

Perform the allocation of processes using-

1. First Fit Algorithm
2. Best Fit Algorithm
3. Worst Fit Algorithm

Solution-

According to question, the main memory has been divided into fixed size partitions as-



Main Memory

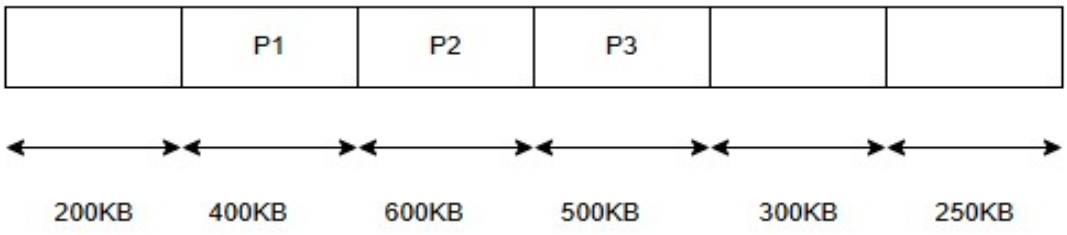
Let us say the given processes are-

- Process P1 = 357 KB
- Process P2 = 210 KB
- Process P3 = 468 KB
- Process P4 = 491 KB

Allocation Using First Fit Algorithm-

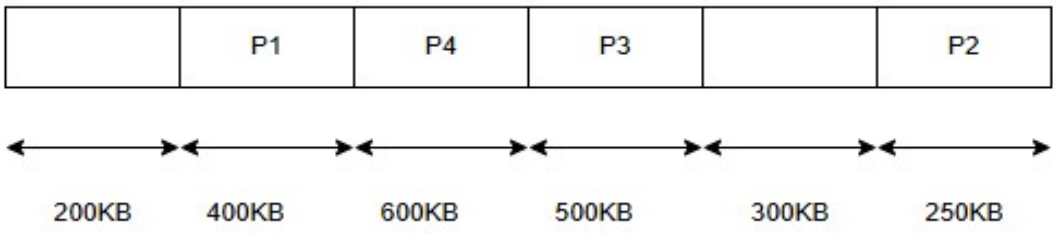
In First Fit Algorithm,

- Algorithm starts scanning the partitions serially.
- When a partition big enough to store the process is found, it allocates that partition to the process.



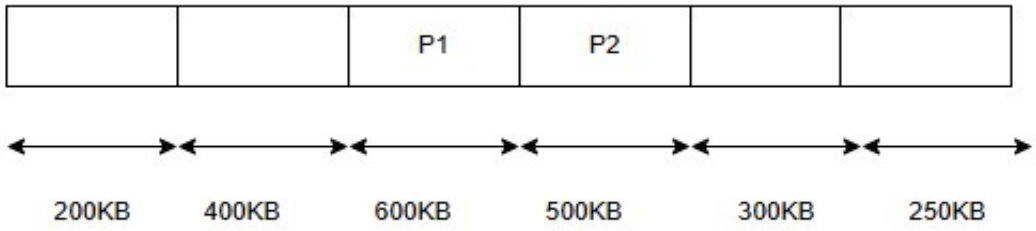
- Process P4 cannot be allocated the memory.
- This is because no partition of size greater than or equal to the size of process P4 is available.

In Best Fit Algorithm,



All processes are allocated to the memory.

In Worst Fit Algorithm,



- Process P3,P4 cannot be allocated the memory.
- This is because no partition of size greater than or equal to the size of process P3, P4 is available.

8. Virtual Memory

A computer system has a limited amount of memory. Adding more memory physically is very costly. Therefore most modern computers use a combination of both hardware and software to allow the computer to address more memory than the amount physically present on the system. This extra memory is actually called **Virtual Memory.**

Virtual Memory is a storage allocation scheme used by the Memory Management Unit(MMU) to compensate for the shortage of physical memory by transferring data from RAM to disk storage. It addresses secondary memory as though it is a part of the main memory. Virtual Memory

makes the memory appear larger than actually present which helps in the execution of programs that are larger than the physical memory.

Virtual Memory can be implemented using two methods:

- Paging
- Segmentation

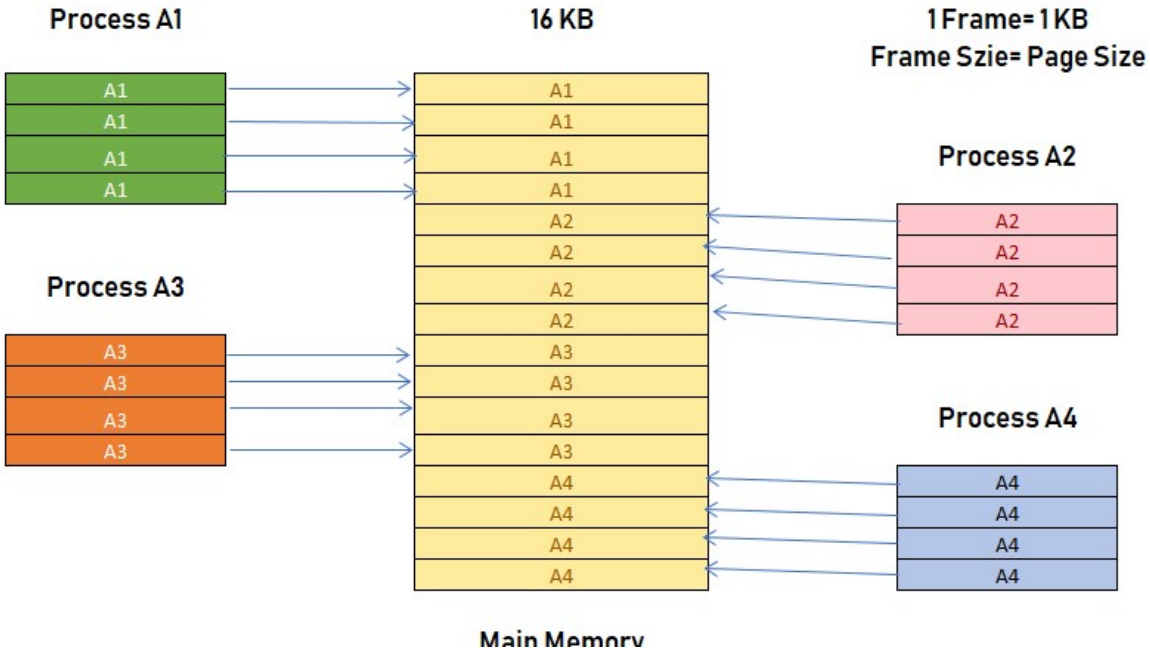
9. Paging in OS

- Paging is a non-contiguous memory allocation technique.
- The logical address generated by the CPU is translated into the physical address using the page table.

Paging is a storage mechanism that allows OS to retrieve processes from the secondary storage into the main memory in the form of pages. In the Paging method, the main memory is divided into small fixed-size blocks of physical memory, which is called frames. The size of a frame should be kept the same as that of a page to have maximum utilization of the main memory and to avoid external fragmentation. Paging is used for faster access to data, and it is a logical concept.

Example of Paging in OS

If the main memory size is 16 KB and Frame size is 1 KB. Here, the main memory will be divided into the collection of 16 frames of 1 KB each. There are 4 separate processes in the system that is A1, A2, A3, and A4 of 4 KB each. Here, all the processes are divided into pages of 1 KB each so that operating system can store one page in one frame. At the beginning of the process, all the frames remain empty so that all the pages of the processes will get stored in a contiguous way



In this example you can see that A2 and A4 are moved to the waiting state after some time. Therefore, eight frames become empty, and so other pages can be loaded in that empty blocks. The process A5 of size 8 pages (8 KB) is waiting in the ready queue.

10. Page Table

Page Table is a data structure used by the virtual memory system to store the mapping between logical addresses and physical addresses.

Logical addresses are generated by the CPU for the pages of the processes therefore they are generally used by the processes.

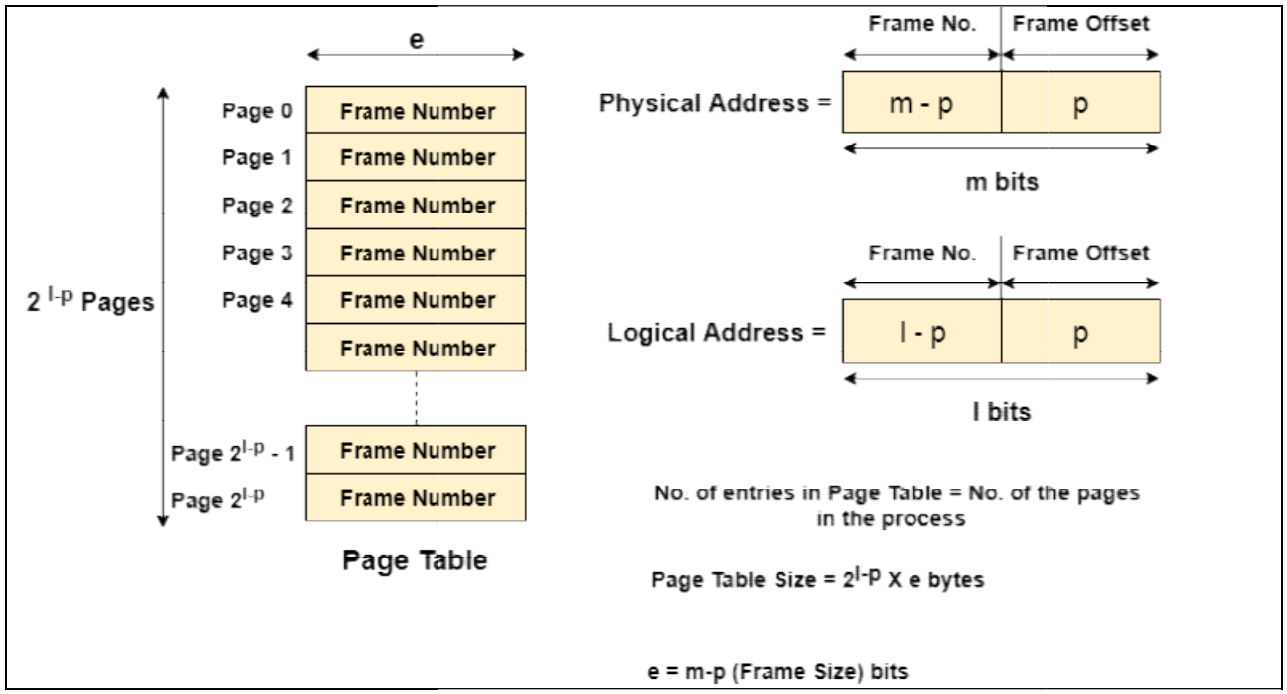
Physical addresses are the actual frame address of the memory. They are generally used by the hardware or more specifically by RAM subsystems.

The image given below considers,

Physical Address Space = M words
Logical Address Space = L words
Page Size = P words

Physical Address = log 2 M = m bits

Logical Address = log 2 L = l bits
page offset = log 2 P = p bits



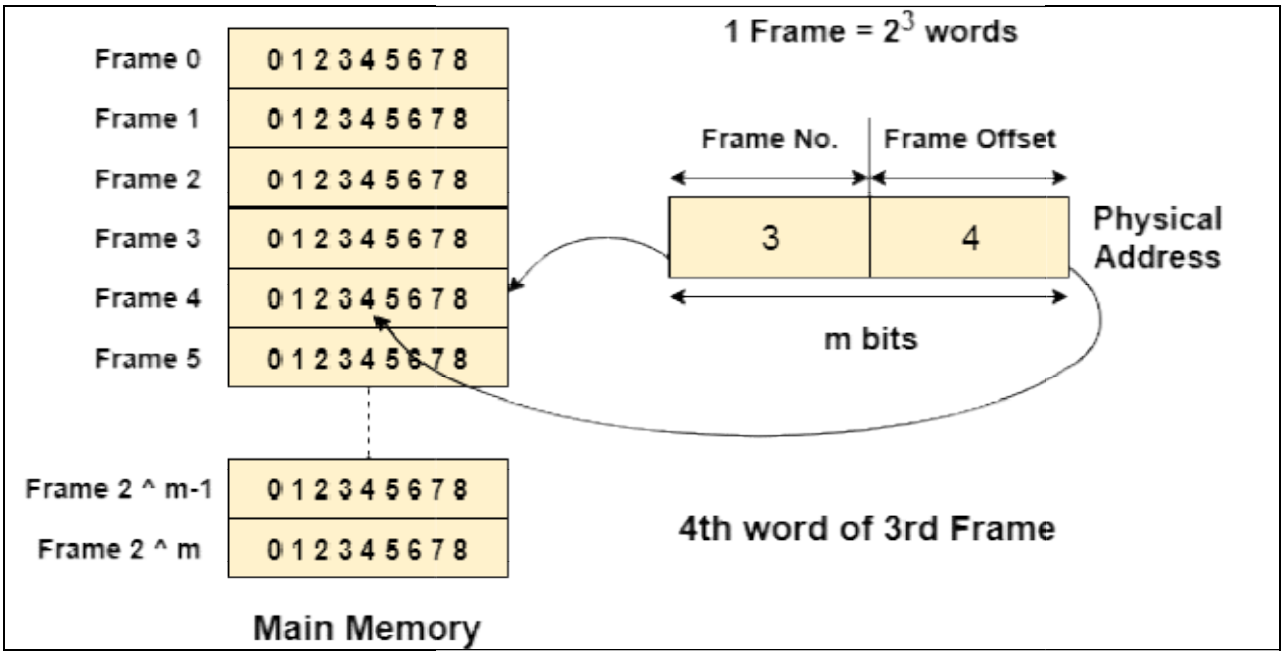
The CPU always accesses the processes through their logical addresses. However, the main memory recognizes physical address only.

In this situation, a unit named as Memory Management Unit comes into the picture. It converts the page number of the logical address to the frame number of the physical address. The offset remains same in both the addresses.

To perform this task, Memory Management unit needs a special kind of mapping which is done by page table. The page table stores all the Frame numbers corresponding to the page numbers of the page table.

In other words, the page table maps the page number to its actual location (frame number) in the memory.

In the image given below shows, how the required word of the frame is accessed with the help of offset.



11. Demand Paging

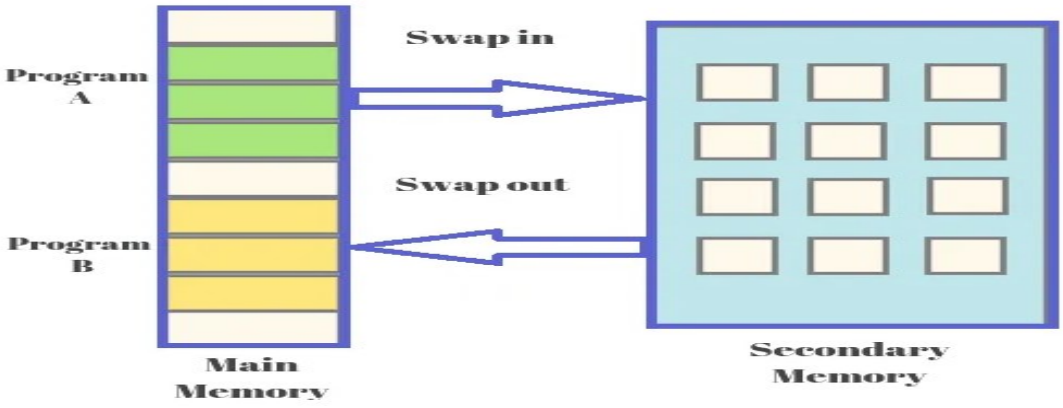
According to the concept of Virtual Memory, in order to execute some process, only a part of the process needs to be present in the main memory which means that only a few pages will only be present in the main memory at any time.

However, deciding, which pages need to be kept in the main memory and which need to be kept in the secondary memory, is going to be difficult because we cannot say in advance that a process will require a particular page at particular time. Therefore, to overcome this problem, there is a concept called Demand Paging is introduced. It suggests keeping all pages of the frames in the secondary memory until they are required. In other words, it says that do not load any page in the main memory until it is required.

Whenever any page is referred for the first time in the main memory, then that page will be found in the secondary memory.

After that, it may or may not be present in the main memory depending upon the page replacement algorithm which will be covered later in this tutorial.

Demand paging is a process of swapping in the Virtual Memory system. In this process, all data is not moved from hard drive to main memory because while using this demand paging, when some programs are getting demand then data will be transferred. But, if required data is already existed into memory then not need to copy of data. The demand paging system is done with swapping from auxiliary storage to primary memory, so it is known as "Lazy Evaluation".



12. Page Fault

If the referred page is not present in the main memory then there will be a miss and the concept is called Page miss or page fault.

The CPU has to access the missed page from the secondary memory. If the number of page fault is very high then the effective access time of the system will become very high.

13. Segmentation

Segmentation method works almost similarly to paging, only difference between the two is that segments are of variable-length whereas, in the paging method, pages are always of fixed size.

A program segment includes the program's main function, data structures, utility functions, etc. The OS maintains a segment map table for all the processes. It also includes a list of free memory blocks along with its size, segment numbers, and its memory locations in the main memory or virtual memory.

13.1 Advantages of Segmentation

- Offer protection within the segments
- You can achieve sharing by segments referencing multiple processes.
- Not offers internal fragmentation
- Segment tables use lesser memory than paging

13.2 Disadvantages of Segmentation

- In segmentation method, processes are loaded/ removed from the main memory. Therefore, the free memory space is separated into small pieces which may create a problem of external fragmentation
- Costly memory management algorithm

14. Page Replacement in Operating System

Page Replacement Definition – Page replacement policies decides that which type of page should be replaced, but these page replacements.

Strategies are implemented when requested page is not existed into primary memory.

Page Replacement Algorithms play vital role in the virtual memory management, because on the base of those Pages replacement policies can be specified that which memory block (page) should be swap out, arising memory space for needed page. Main objective of all Page replacement policies are to decrease the maximum number of page faults.

15. Types of Page Replacement Algorithms

There are various page replacement techniques:

A. FIFO Page Replacement Algorithm
B. LIFO Page Replacement Algorithm
C. LRU Page Replacement Algorithm
D. Optimal Page Replacement Algorithm
E. Random Page Replacement Algorithm

15.1 FIFO Page Replacement Algorithm

This page replacement algorithm is very easy and simple because this algorithm is based on the "First in First out "principle. As per FIFO principle, oldest page is replaced at the front side and most recent page is replaced at the rear side.

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

For Example: Consider the page reference string of size 12: 1, 2, 3, 4, 5, 1, 3, 1, 6, 3, 2, 3 with

1	2	3	4	5	1	3	1	6	3	2	3

1	1	1	1	5	5	5	5	5	5	2	2
	2	2	2	2	1	1	1	1	1	1	1
		3	3	3	3	3	3	6	6	6	6
			4	4	4	4	4	4	3	3	3
M	M	M	M	M	M	H	H	M	M	M	H

M = Miss
H = Hit
frame size 4(i.e. maximum 4 pages in a frame).

Total Page Fault = 9
Initially, all 4 slots are empty, so when 1, 2, 3, 4 came they are allocated to the empty slots in order of their arrival. This is page fault as 1, 2, 3, 4 are not available in memory.
When 5 come, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 1.
When 1 comes, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 2.
When 3, 1 comes; it is available in the memory, i.e., Page Hit, so no replacement occurs.

When 6 come, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 3.
When 3 come, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 4.
When 2 come, it is not available in memory so page fault occurs and it replaces the oldest page in memory, i.e., 5.
When 3 come, it is available in the memory, i.e., Page Hit, so no replacement occurs.
Page Fault ratio = 9/12 i.e. total miss/total possible cases

Advantages of FIFO Page Replacement

- It uses simple method, and easy to use.
- It does not give more overhead.

Disadvantages of FIFO Page Replacement

- Worst performance
- Don't use the frequency of last used time, just replace the oldest page.
- Getting increase the page faults, while increasing page frames.

15.2 LRU Page Replacement Algorithm

LRU stands for **"Least recently used"**, and it helps to operating system for searching such page that is used over the short duration of time frame. This page replacement algorithm uses the counter along with even page, and that counter is known as aging registers.

LRU algorithm helps to select that page which is not needed for long life in to primary memory. Least Recently Used page replacement algorithm keeps track of page usage over a short period of time. It works on the idea that the pages that have been most heavily used in the past are most likely to be used heavily in the future too.

In LRU, whenever page replacement happens, the page which has not been used for the longest amount of time is replaced.

For Example

1	2	3	4	5	1	3	1	6	3	2	3
1	1	1	1	5	5	5	5	5	5	2	2
	2	2	2	2	1	1	1	1	1	1	1
		3	3	3	3	3	3	3	3	3	3
			4	4	4	4	4	6	6	6	6
M	M	M	M	M	M	H	H	M	H	M	H

M = Miss
H = Hit

Total Page Fault = 8

Initially, all 4 slots are empty, so when 1, 2, 3, 4 came they are allocated to the empty slots in order of their arrival. This is page fault as 1, 2, 3, 4 are not available in memory.

When 5 come, it is not available in memory so page fault occurs and it replaces 1 which is the least recently used page.

When 1 comes, it is not available in memory so page fault occurs and it replaces 2.

When 3, 1 comes; it is available in the memory, i.e., Page Hit, so no replacement occurs.

When 6 come, it is not available in memory so page fault occurs and it replaces 4.

When 3 come, it is available in the memory, i.e., Page Hit, so no replacement occurs.

When 2 come, it is not available in memory so page fault occurs and it replaces 5.

When 3 come, it is available in the memory, i.e., Page Hit, so no replacement occurs.

Page Fault ratio = 8/12

15.3 Optimal Page Replacement Algorithm

Optimal Page Replacement Algorithm is very excellent page replacement policy because it helps to provide least number of page faults, so it is called of "OPT", "Clairvoyant Replacement Algorithm", and "Belady's optimal page policy".

15.4 LIFO Page Replacement Algorithm

LIFO stands for "Last in First out", and it performs all activities like LIFO principle. In this algorithm, newest page is replaced which is arrived at last in to primary memory, and it uses the stack for monitoring all pages.

16. ALLOCATION OF FRAMES

When a page fault occurs, there is a free frame available to store new page into a frame. While the page swap is taking place, a replacement can be selected, which is written to the disk as the user process continues to execute. The operating system allocates all its buffer and table space from the free-frame list for new page.

Two major allocation Algorithm/schemes

1. Equal allocation

2. Proportional allocation

1. Equal allocation: The easiest way to split m frames among n processes is to give everyone an equal share, m/n frames. This scheme is called equal allocation.

2. Proportional allocation: Here, it allocates available memory to each process according to its size. Let the size of the virtual memory for process pi be si, and define $S= \sum Si$

Then, if the total number of available frames is m, we allocate ai frames to process pi, where

ai is approximately $ai = Si/ S \times m$.

UNIT RELATED QUESTION

Q.1 Define: i) Physical Address Space, ii) Logical Address Space, iii) Static Loading, iv) Dynamic Loading, v) Swapping, vi) Segmentation, vii) Page Fault, viii) Page replacement in OS, ix) Page Table

Q.2 Write short note on memory management and requirement of it.

Q.3 Write short note on virtual memory.

Q.4 Explain static partitioning including First fit algorithm, Worst fit algorithm, and Best fit algorithm in detail.

Q.5 Consider six memory partitions of size 50 K, 75 K, 150 K, 500 KB, 175 K and 300 K. These partitions need to be allocated to four processes of sizes 25 K, 50 K, 100 K and 75 K in that order. Perform the allocation of processes using- 1) first fit Algorithm, 2) Best Fit Algorithm, 3) Worst Fit Algorithm.

Q.6 What do you mean by paging in OS. Explain its example in detail including diagram.

Q.7 Write a short note on page table including diagram.

Q.8 Write a short note on segmentation including its advantages and disadvantages.

Q.9 Write a short note on demand paging including diagram.

Q.10 What do you mean by page replacement. Write its types and explain any one of them in detail with example.

Q.11 Explain Optimal Page Replacement Algorithm in detail with example table.

Q.12 Explain LRU Algorithm in detail with example table.

Q.13 Explain FIFO Page Replacement Algorithm in detail with example table.

Q.14 Write short note on allocation of frames.

UNIT -4
Index

1. File System Interface

The file system provides the mechanism for on-line storage of and access to both data and programs of the operating system and all the users of the computer system. The file system consists of two distinct parts: a collection of files, each storing related data, and a directory structure, which organizes and provides information about all the files in the system.

2. File Concept

A file is a collection of related information that is recorded on secondary storage. From a user's perspective, a file is the smallest allotment of logical secondary storage and data cannot be written to secondary storage unless they are within a file.

Four terms are in common use when discussing files: Field, Record, File and Database

⇒A field is the basic element of data. An individual field contains a single value, such as an employee's last name, a date, or the value of a sensor reading. It is characterized by its length and data type.

⇒A record is a collection of related fields that can be treated as a unit by some application program. For example, an employee record would contain such fields as name, social security number, job classification, date of hire, and so on.

⇒A file is a collection of similar records. The file is treated as a single entity by users and applications and may be referenced by name.

⇒A database is a collection of related data. A database may contain all of the information related to an organization or project, such as a business or a scientific study. The database itself consists of one or more types of files.

3. File Attributes

A file has the following attributes:

⇒**Name:** The symbolic file name is the only information kept in human readable form.

⇒**Identifier**: This unique tag, usually a number, identifies the file within the file system; it is the non-human-readable name for the file.

⇒**Type:** This information is needed for those systems that support different types.

⇒**Location**: This information is a pointer to a device and to the location of the file on that device.

⇒**Size:** The current size of the file (in bytes, words, or blocks), and possibly the maximum allowed size are included in this attribute.

⇒**Protection:** Access-control information determines who can do reading, writing, executing, and so on.

⇒**Time, date, and user identification:** This information may be kept for creation, modification and last use. These data can be useful for protection, security, and usage monitoring.

4. File Operations

The operating system can provide system calls to create, write, read, reposition, delete, and truncate files. The file operations are described as followed:

- **Creating a file:** Two steps are necessary to create a file. First, space in the file system must be found for the file. Second, an entry for the new file must be made in the directory. The directory entry records the name of the file and the location in the file system, and possibly other information.

- **Writing a file:** To write a file, we make a system call specifying both the name of the file and the information to be written to the file. Given the name of the file, the system searches the directory to find the location of the file. The system must keep a write pointer to the location in the file where the next write is to take place. The write pointer must be updated whenever a write occurs.

- **Reading a file:** To read from a file, we use a system call that specifies the name of the file and where (in main memory) the next block of the file should be put. Again, the directory is searched for the associated directory entry, and the system needs to keep a read pointer to the location in the file where the next read is to take place. Once the read has taken place, the read pointer is updated.

- **Repositioning within a file:** The directory is searched for the appropriate entry, and the current-file-position is set to a given value. Repositioning within a file does not need to involve any actual I/O. This file operation is also known as a file seeks.

- **Deleting a file:** To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space, so that it can be reused by other files, and erase the directory entry.
- **Truncating a file:** The user may want to erase the contents of a file but keep its attributes. Rather than forcing the user to delete the file and then recreate it, this function allows all attributes to remain unchanged-except for file length-but lets the file be reset to length zero and its file space released.

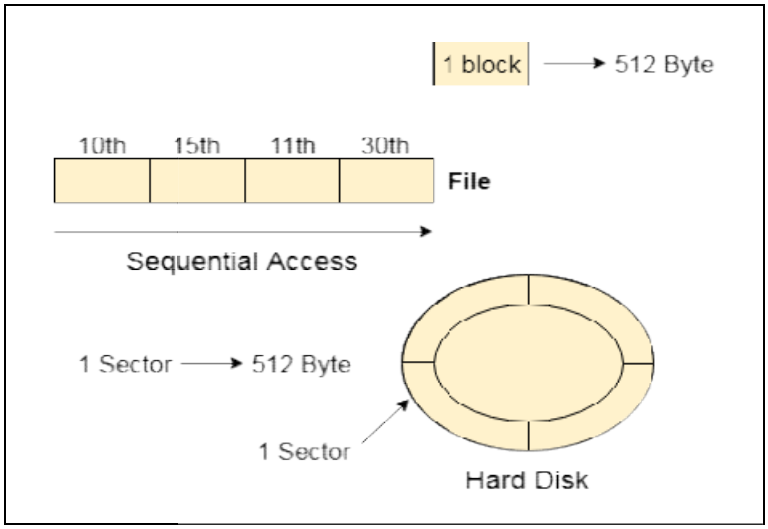
5. File Types: The files are classified into different categories as follows

file type	usual extension	function
executable	exe, com, bin or none	read to run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rrf, doc	various word-processor formats
library	lib, a, so, dll, mpeg, mov, rm	libraries of routines for programmers
print or view	arc, zip, tar	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com-pressed, for archiving or storage
multimedia	mpeg, mov, rm	binary file containing audio or A/V information

6. File Access Methods

Let's look at various ways to access files stored in secondary memory.

i. Sequential Access



Most of the operating systems access the file sequentially. In other words, we can say that most of the files need to be accessed sequentially by the operating system.

In sequential access, the OS read the file word by word. A pointer is maintained which initially points to the base address of the file. If the user wants to read first word of the file then the pointer provides that word to the user and increases its value by 1 word. This process continues till the end of the file.

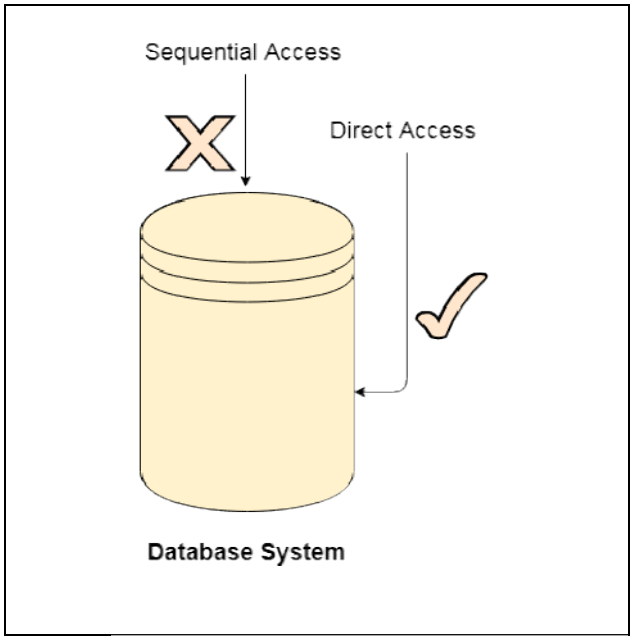
Modern word systems do provide the concept of direct access and indexed access but the most used method is sequential access due to the fact that most of the files such as text files, audio files, video files, etc need to be sequentially accessed.

ii. Direct Access

The Direct Access is mostly required in the case of database systems. In most of the cases, we need filtered information from the database. The sequential access can be very slow and inefficient in such cases.

Suppose every block of the storage stores 4 records and we know that the record we needed is stored in 10th block. In that case, the sequential access will not be implemented because it will traverse all the blocks in order to access the needed record.

Direct access will give the required result despite of the fact that the operating system has to perform some complex tasks such as determining the desired block number. However, that is generally implemented in database applications.



iii. Indexed Access

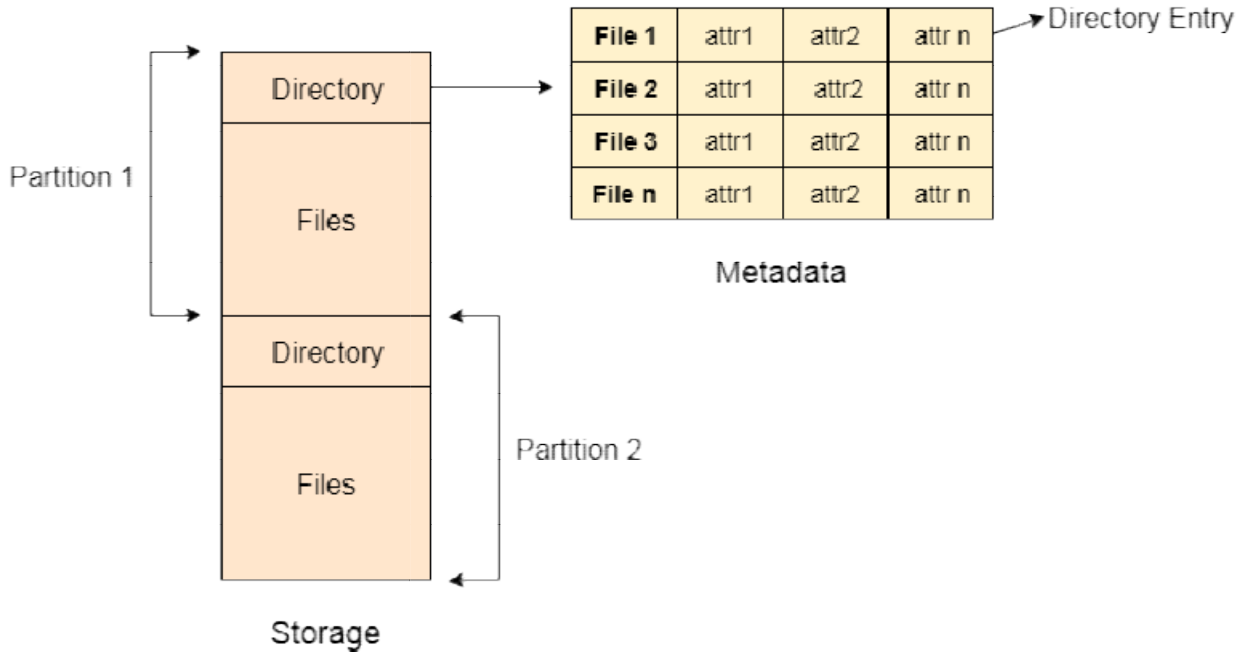
If a file can be sorted on any of the filed then an index can be assigned to a group of certain records. However, A particular record can be accessed by its index. The index is nothing but the address of a record in the file. In index accessing, searching in a large database became very quick and easy but we need to have some extra space in the memory to store the index value.

7. Directory Structure

Directory can be defined as the listing of the related files on the disk. The directory may store some or the entire file attributes.

To get the benefit of different file systems on the different operating systems, a hard disk can be divided into the number of partitions of different sizes. The partitions are also called volumes or mini disks.

Each partition must have at least one directory in which, all the files of the partition can be listed. A directory entry is maintained for each file in the directory which stores all the information related to that file.



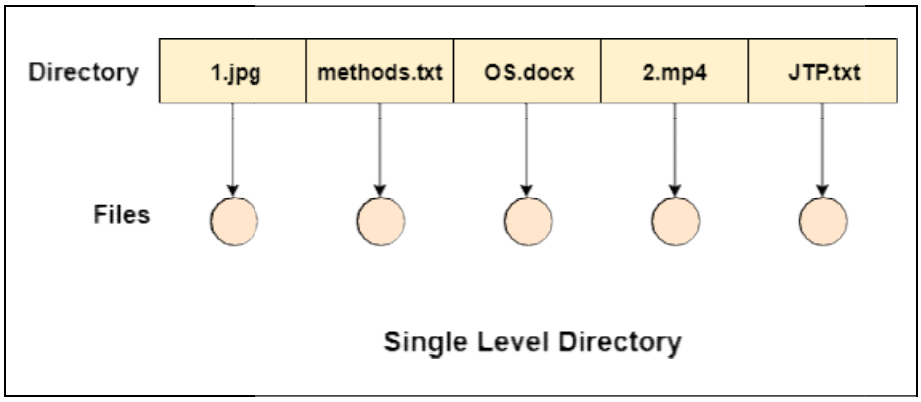
A directory can be viewed as a file which contains the Meta data of the bunch of files.

Every Directory supports a number of common operations on the file:

1. File Creation
2. Search for the file
3. File deletion
4. Renaming the file
5. Traversing Files
6. Listing of file

i. Single Level Directory

The simplest method is to have one big list of all the files on the disk. The entire system will contain only one directory which is supposed to mention all the files present in the file system. The directory contains one entry per each file present on the file system.



This type of directories can be used for a simple system.

Advantages

1. Implementation is very simple.
2. If the sizes of the files are very small then the searching becomes faster.
3. File creation, searching, deletion is very simple since we have only one directory.

Disadvantages

i. We cannot have two files with the same name.

ii. The directory may be very big therefore searching for a file may take so much time.

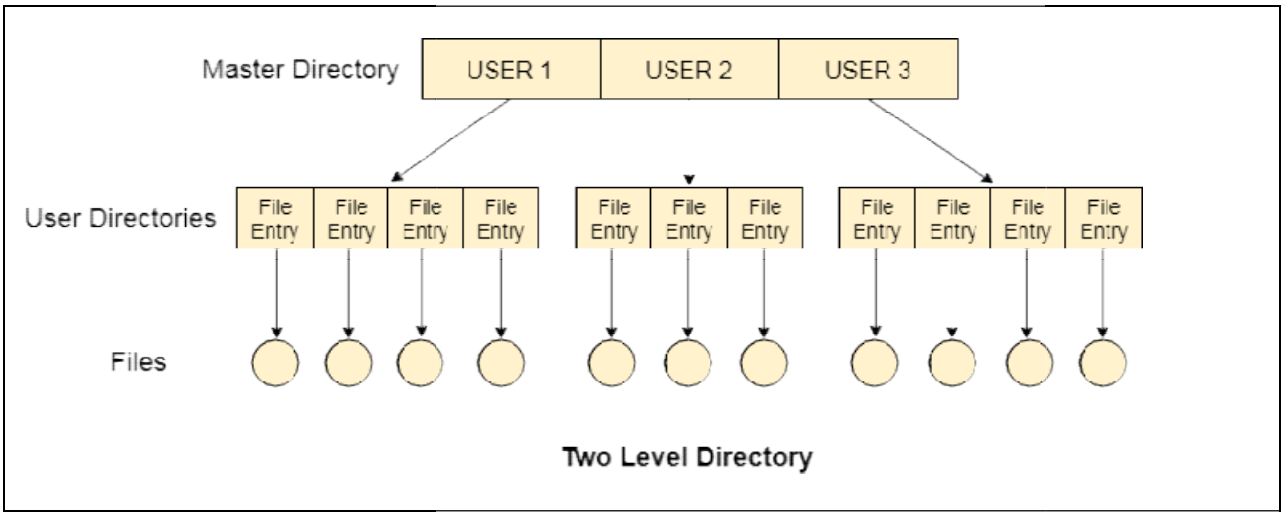
iii. Protection cannot be implemented for multiple users.

iv. There are no ways to group same kind of files.

v. Choosing the unique name for every file is a bit complex and limits the number of files in the system because most of the Operating System limits the number of characters used to construct the file name.

ii. Two Level Directory

In two level directory systems, we can create a separate directory for each user. There is one master directory which contains separate directories dedicated to each user. For each user, there is a different directory present at the second level, containing group of user's file. The system doesn't let a user to enter in the other user's directory without permission.



Characteristics of two level directory system

1. Each files has a path name as */User-name/directory-name/*
2. Different users can have the same file name.
3. Searching becomes more efficient as only one user's list needs to be traversed.
4. The same kind of files cannot be grouped into a single directory for a particular user.

Every Operating System maintains a variable as **PWD** which contains the present directory name (present user name) so that the searching can be done appropriately.

8. File System Mounting

Mounting refers to the grouping of files in a file system structure accessible to the user of the group of users. It can be local or remote, in the local mounting; it connects disk drivers as one machine, while in the remote mounting it uses Network File System (NFS) to connect to directories on other machines so that they can be used as if they are the part of the user's file system.

The directory structure can be built out of multiple volumes which are supposed to be mounted to make them available within the file-system namespace. The procedure for mounting is simple; the OS is given the name of the device and the location within the file structure where the file system is attached.

For example, in a UNIX system, there is a single directory tree, and all the accessible storage must have a location in the single directory tree. Mounting is used to make the storage accessible. A file system containing the user's home directories might be mounted as /home, and they can be accessed by using directory names with time like /home/janc. Similarly, if the file system is mounted as /user, then we will use /user/janc to access it. Then the operating system verifies if the device contains a valid file system by asking the device driver to read the directory and verify that the directory has the expected format. Then the operating system finally notes down the directory structure that the file system is mounted at the specified mount point.

This method helps the operating system traverse through the directory structure and switch among file systems as appropriate.

A system may either allow the same file system to be mounted repeatedly on different mount points or it may allow one mount per file system.

For example, the Macintosh operating system. In this whenever the system encounters a disk for the first time, it searches for the file system in the disk, and if it finds one it automatically mounts the system at the root level and adds a folder icon on the screen labelled as the name of the file system. The Microsoft OS maintains a two-level directory structure.

9. File Sharing in Operating System

File sharing, also known as **file-swapping** is the accessing or sharing of files by one or more users. It is performed on computer networks as a quick way to transmit data. Generally, a file-sharing system usually has more than one administrator, where the users may have the same or different access privileges. It also implies having an allocated amount of personal files in the common storage.

File sharing has been used in mainframe and multi-user computer systems for many years, and now with widespread access to the internet, a file transfer system known as the **File-Transfer Protocol** or FTP is widely used.

The World Wide Web in itself can be considered as a large-scale file-sharing system where files are constantly downloaded or viewed by the web user.

However, in the file-sharing system, the user can read and write in the files, and if the entire system is not available for access, at least the common file that is shared by the owner can be read and written onto.

Certain issues arise when multiple users are allowed to access the files. The system can either grant access to the users by default or requires the user to specifically grant access to the files. This causes problems of control and protection, and to implement sharing and protection, the system needs to maintain more file and directory attributes than it would have done in the case of a single-user system.

10. Protection in Operating System

Protection is especially important in a multiuser environment when multiple users use computer resources such as CPU, memory, etc. It is the operating system's responsibility to offer a mechanism that protects each process from other processes. In a multiuser environment, all assets that require protection are classified as objects, and those that wish to access these objects are referred to as subjects. The operating system grants different 'access rights' to different subjects.

A mechanism that controls the access of programs, processes, or users to the resources defined by a computer system is referred to as protection. You may utilize protection as a tool for multi-programming operating systems, allowing multiple users to safely share a common logical namespace, including a directory or files.

It needs the protection of computer resources like the software, memory, processor, etc. Users should take protective measures as a helper to multiprogramming OS so that multiple users may safely use a common logical namespace like a directory or data. Protection may be achieved by maintaining confidentiality, honesty and availability in the OS. It is critical to secure the device from unauthorized access, viruses, worms, and other malware.

10.1 Need of Protection in Operating System

Various needs of protection in the operating system are as follows:

1. There may be security risks like unauthorized reading, writing, modification, or preventing the system from working effectively for authorized users.

2. It helps to ensure data security, process security, and program security against unauthorized user access or program access.

3. It is important to ensure no access rights' breaches, no viruses, no unauthorized access to the existing data.

4. Its purpose is to ensure that only the systems' policies access programs, resources, and data.

10.2 Goals of Protection in Operating System

Various goals of protection in the operating system are as follows:

1. The policies define how processes access the computer system's resources, such as the CPU, memory, software, and even the operating system. It is the responsibility of both the operating system designer and the app programmer. Although, these policies are modified at any time.

2. Protection is a technique for protecting data and processes from harmful or intentional infiltration. It contains protection policies either established by itself, set by management or imposed individually by programmers to ensure that their programs are protected to the greatest extent possible.

3. It also provides a multiprogramming OS with the security that its users expect when sharing common space such as files or directories.

10.3 Role of Protection in Operating System

Its main role is to provide a mechanism for implementing policies that define the use of resources in a computer system. Some rules are set during the system's design, while others are defined by system administrators to secure their files and programs.

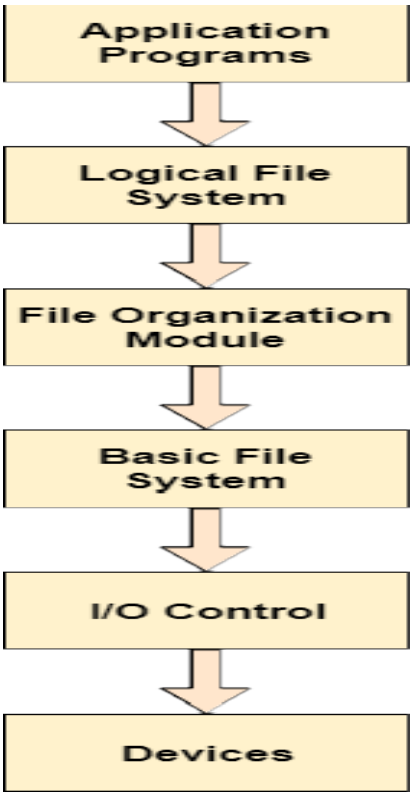
Every program has distinct policies for using resources, and these policies may change over time. Therefore, system security is not the responsibility of the system's designer, and the programmer must also design the protection technique to protect their system against infiltration.

11. File System Structure

File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way. A file System must be able to store the file, locate the file and retrieve the file.

Most of the Operating Systems use layering approach for every task including file systems. Every layer of the file system is responsible for some activities.

The image shown below, elaborates how the file system is divided in different layers, and also the functionality of each layer.



- When an application program asks for a file, the first request is directed to the logical file system. The logical file system contains the Meta data of the file and directory structure. If the application program doesn't have the required permissions of the file then this layer will throw an error. Logical file systems also verify the path to the file.

- Generally, files are divided into various logical blocks. Files are to be stored in the hard disk and to be retrieved from the hard disk. Hard disk is divided into various tracks and sectors. Therefore, in order to store and retrieve the files, the logical blocks need to be mapped to physical blocks. This mapping is done by File organization module. It is also responsible for free space management.

- Once File organization module decided which physical block the application program needs, it passes this information to basic file system. The basic file system is responsible for issuing the commands to I/O control in order to fetch those blocks.

- I/O controls contain the codes by using which it can access hard disk. These codes are known as device drivers. I/O controls are also responsible for handling interrupts.

12. Mass Storage Structure - DISK STRUCTURE

Magnetic disks provide the bulk of secondary storage for modern computer systems. Each disk platter has a flat circular shape, like a CD. Common platter diameters range from 1.8 to 5.25 inches. The two surfaces of a platter are covered with a magnetic material. We store information by recording it magnetically on the platters.

A read-write head "flies" just above each surface of every platter. The heads are attached to a disk arm, which moves all the heads as a unit. The surface of a platter is logically divided into circular tracks, which are subdivided into sectors. The set of tracks that are at one arm position forms a cylinder. There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors. The storage capacity of common disk drives is measured in gigabytes.

When the disk is in use, a drive motor spins it at high speed. Most drives rotate 60 to 200 times per second. Disk speed has two parts. The transfer rate is the rate at which data flow between the drive and the computer. The positioning time (or random-access time)consists of seek time and rotational latency. The seek time is the time to move the disk arm to the desired cylinder. And the rotational latency is the time for the desired sector to rotate to the disk head. Typical disks can transfer several megabytes of data per second and they have seek times and rotational latencies of several milliseconds.

13. Disk Scheduling

A process needs two types of time, CPU time and IO time. For I/O, it requests the Operating system to access the disk.

However, the operating system must be fare enough to satisfy each request and at the same time, operating system must maintain the efficiency and speed of process execution.

The technique that operating system uses to determine the request which is to be satisfied next is called disk scheduling.

Some important terms related to disk scheduling.

i. **Seek Time:** Seek time is the time taken in locating the disk arm to a specified track where the read/write request will be satisfied.

ii. **Rotational Latency:** It is the time taken by the desired sector to rotate itself to the position from where it can access the R/W heads.

iii. **Transfer Time:** It is the time taken to transfer the data.

iv. **Disk Access Time** = Rotational Latency + Seek Time + Transfer Time

v. **Disk Response Time:** It is the average of time spent by each request waiting for the IO operation.

14. Purpose of Disk Scheduling

The main purpose of disk scheduling algorithm is to select a disk request from the queue of IO requests and decide the schedule when this request will be processed.

15. Goal of Disk Scheduling Algorithm

- Fairness
- High throughout
- Minimal traveling head time

16. Disk Scheduling Algorithms

The list of various disks scheduling algorithm is given below. Each algorithm is carrying some advantages and disadvantages. The limitation of each algorithm leads to the evolution of a new algorithm.

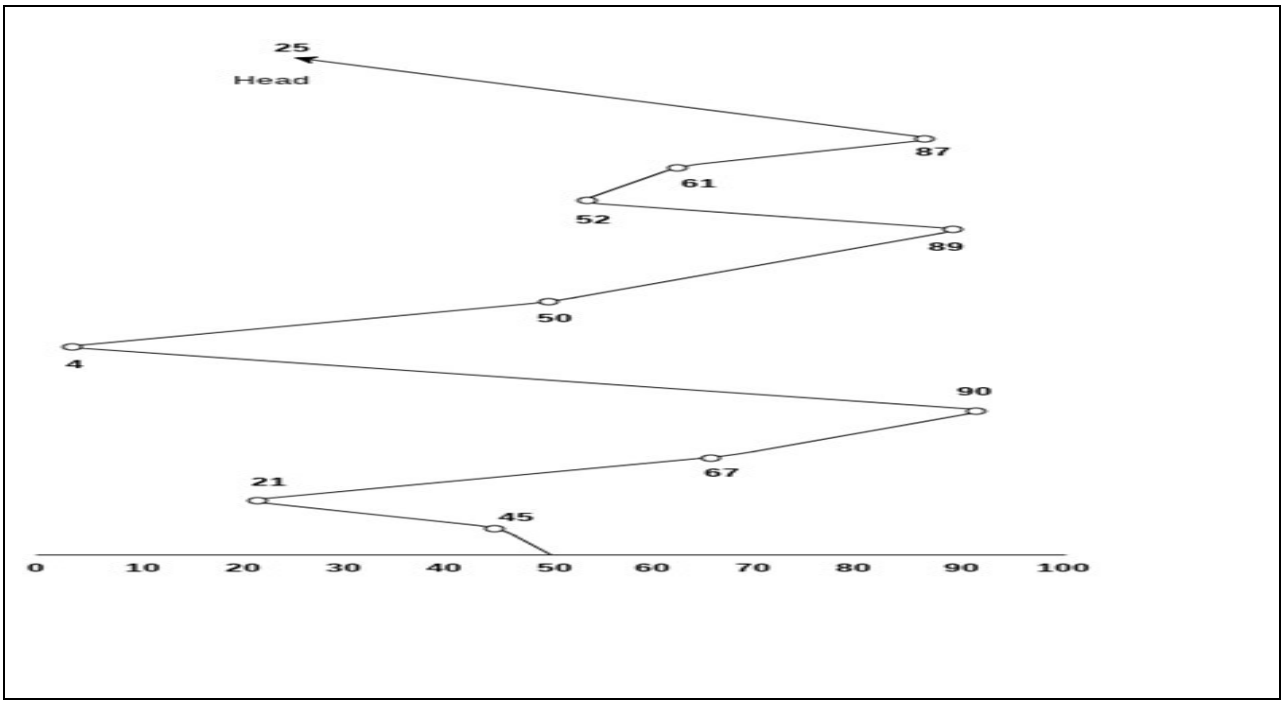
- FCFS scheduling algorithm
- SSTF (shortest seek time first) algorithm
- SCAN scheduling
- C-SCAN scheduling
- LOOK Scheduling
- C-LOOK scheduling

16.1 FCFS Scheduling Algorithm

- It is the simplest Disk Scheduling algorithm. It services the IO requests in the order in which they arrive. There is no starvation in this algorithm, every request is serviced.

- **Consider the following disk request sequence for a disk with 100 tracks 45, 21, 67, 90, 4, 50, 89, 52, 61, 87, 25; Head pointer starting at 50 and moving in left direction. Find the number of head movements in cylinders using FCFS scheduling.**

Solution



Number of cylinders moved by the head

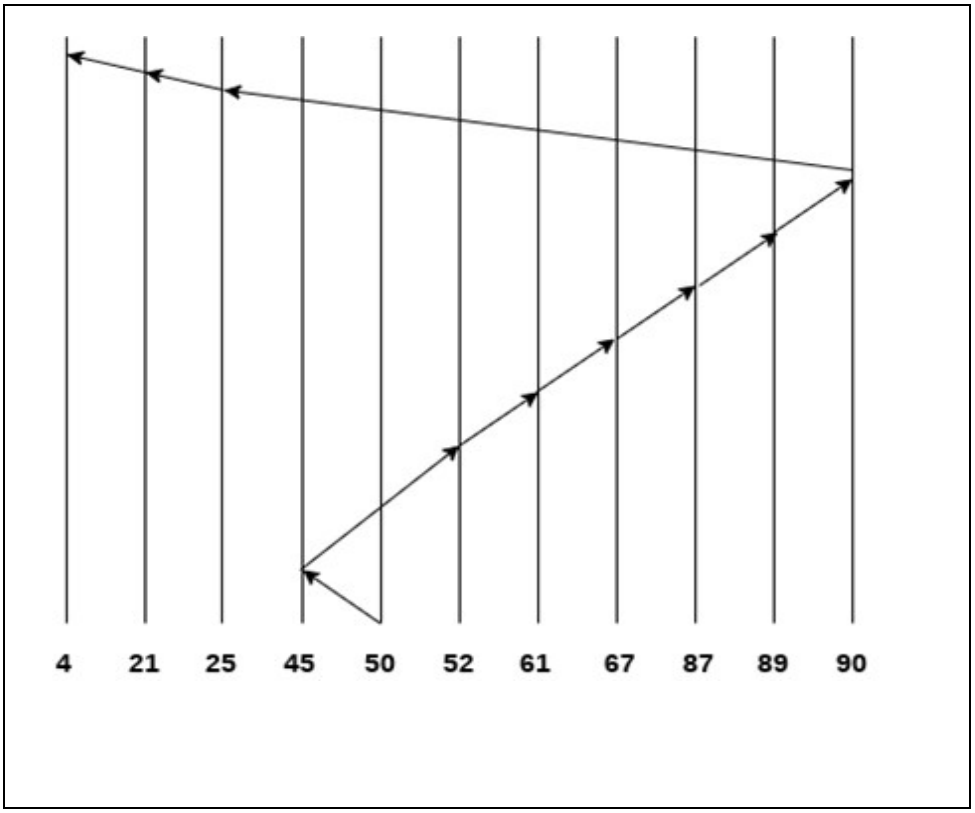
=(50-45)+(45-21)+(67-21)+(90-67)+(90-4)+(50-4)+(89-50)+(61-52)+(87-61)+(87-25)

= 5 + 24 + 46 + 23 + 86 + 46 + 49 + 9 + 26 + 62 = 376

16.2 SSTF (shortest seek time first) algorithm

- Shortest seek time first (SSTF) algorithm selects the disk I/O request which requires the least disk arm movement from its current position regardless of the direction. It reduces the total seek time as compared to FCFS.
- It allows the head to move to the closest track in the service queue.

Example: Consider the following disk request sequence for a disk with 100 tracks: 45, 21, 67, 90, 4, 89, 52, 61, 87, 25; Head pointer starting at 50**. Find the number of head movements in cylinders using SSTF scheduling.**



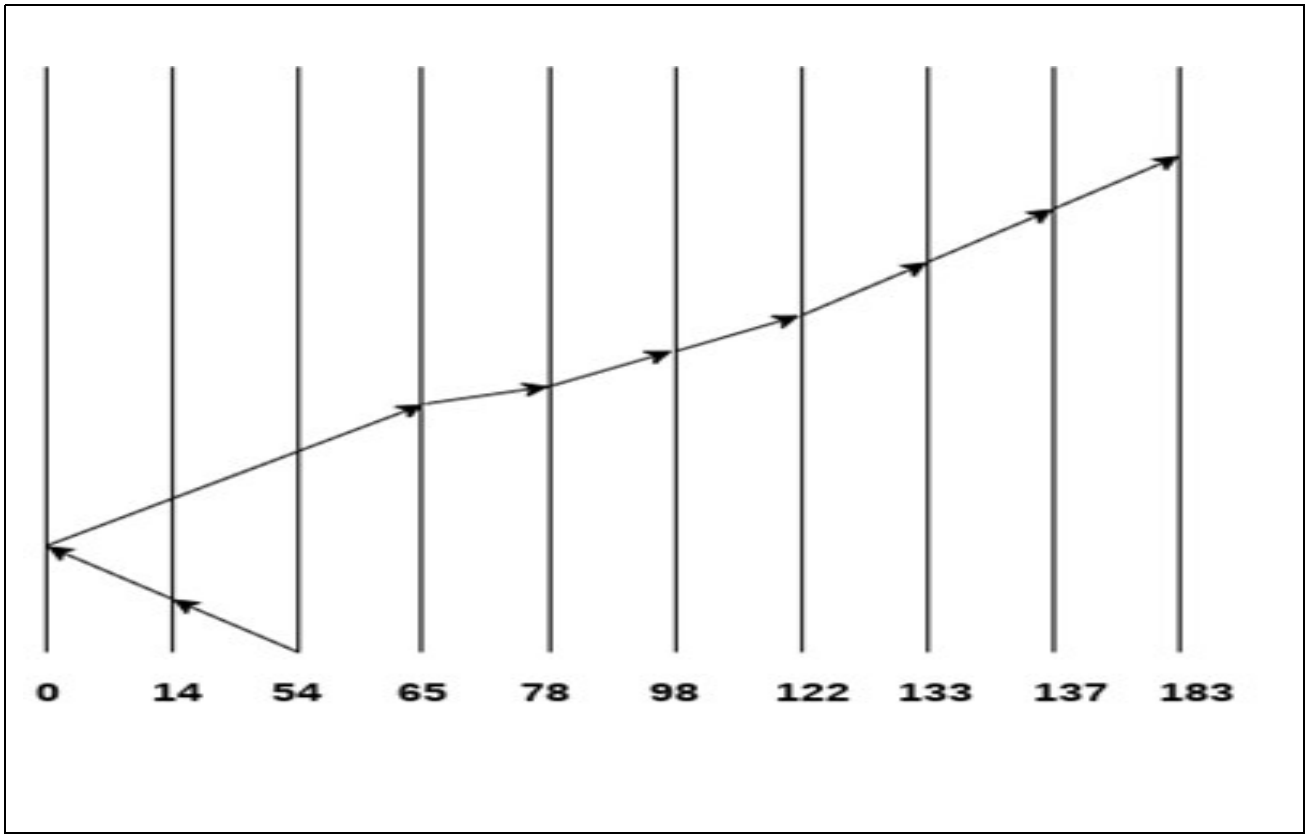
Number of cylinders = 5 + 7 + 9 + 6 + 20 + 2 + 1 + 65 + 4 + 17 = 136

16.3 Scan Algorithm

- It is also called as Elevator Algorithm. In this algorithm, the disk arm moves into a particular direction till the end, satisfying all the requests coming in its path, and then it turns back and moves in the reverse direction satisfying requests coming in its path.

- It works in the way an elevator works, elevator moves in a direction completely till the last floor of that direction and then turns back.

Example - Consider the following disk request sequence for a disk with 100 tracks: 98, 137, 122, 183, 14, 133, 65, 78. Head pointer starting at 54 and moving in left direction. **Find the number of head movements in cylinders using SCAN scheduling.**



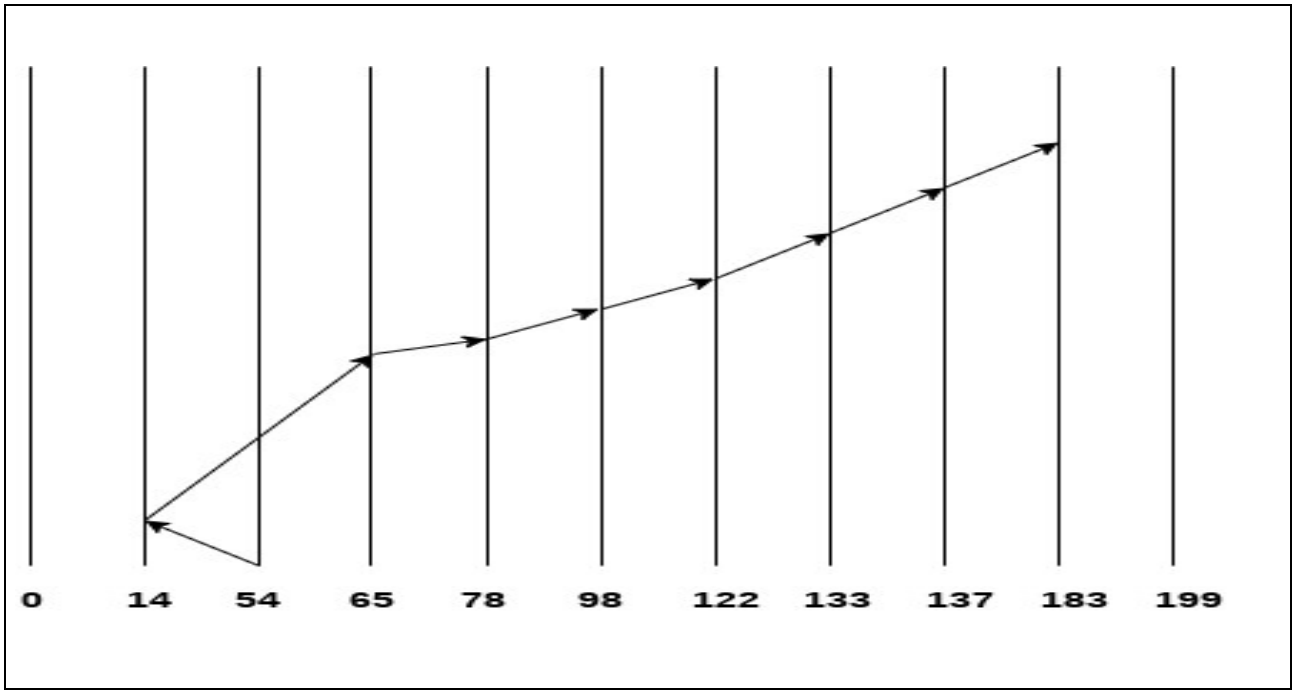
Number of Cylinders = 40 + 14 + 65 + 13 + 20 + 24 + 11 + 4 + 46 = 237

16.4 Look Scheduling

- It is like SCAN scheduling Algorithm to some extant except the difference that, in this scheduling algorithm, the arm of the disk stops moving inwards (or outwards) when no more request in that direction exists. This algorithm tries to overcome the overhead of SCAN algorithm which forces disk arm to move in one direction till the end regardless of knowing if any request exists in the direction or not.

Example

- Consider the following disk request sequence for a disk with 100 tracks
- 98, 137, 122, 183, 14, 133, 65, 78
- Head pointer starting at 54 and moving in left direction. Find the number of head movements in cylinders using LOOK scheduling.



Number of cylinders crossed = 40 + 51 + 13 + +20 + 24 + 11 + 4 + 46 = 209

Unit Related Question

Q.1 Define: i) File System Interface, ii) File concept , iii) File Attribute , iv) File Mounting , v) File sharing in Operating System , vi) Disk Access Time , vii) Seek Time , viii) Disk Response Time , ix) Transfer Time , x) Rotational Latency

Q.2 Explain file operation in detail.

Q.3 Write a short note on file system structure with its diagram

Q.4 Write a short note on directory structure with its diagram.

Q.5 Write a short note on single level directory with its diagram.

Q.6 Write a short note on two level directory with its diagram.

Q.7 Explain each type of file access (Sequential Access, Direct Access, Indexed Access) method including its diagram.

Q.8 Explain protection in operating system including its role goal and need of it in detail.

Q.9 Write a short note on disk structure.

Q.10 Explain disk scheduling including its important term with definition in detail.

Q.11 Explain FCFS (First Come First Serve) disk algorithm with example and diagram in detail.

Q.12 Explain SCAN disk algorithm with example and diagram in detail.

Q.13 Explain SSTF (Shortest Seek Time First) disk algorithm with example and diagram in detail.

Q.14 Explain look scheduling disk algorithm with example and diagram in detail.

Q.15 What do you mean by disk scheduling algorithm. Mention its types.

UNIT -5

Index

1 Deadlock

Every process needs some resources to complete its execution. However, the resource is granted in a sequential order.

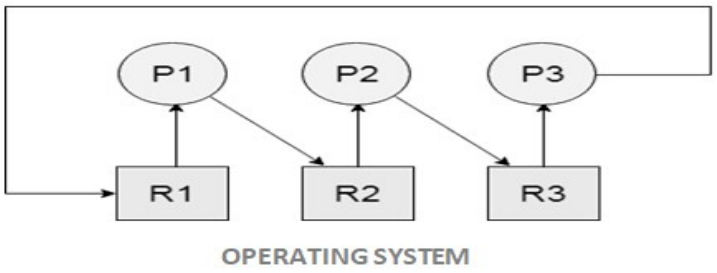
1. The process requests for some resource.
2. OS grant the resource if it is available otherwise let the process waits.
3. The process uses it and release on the completion.

A Deadlock is a situation where each of the computer process waits for a resource which is being assigned to some another process. In this situation, none of the process gets executed since the resource it needs, is held by some other process which is also waiting for some other resource to be released.

Let us assume that there are three processes P1, P2 and P3. There are three different resources R1, R2 and R3. R1 is assigned to P1, R2 is assigned to P2 and R3 is assigned to P3.

After some time, P1 demands for R1 which is being used by P2. P1 halts its execution since it can't complete without R2. P2 also demands for R3 which is being used by P3. P2 also stops its execution because it can't continue without R3. P3 also demands for R1 which is being used by P1 therefore P3 also stops its execution.

In this scenario, a cycle is being formed among the three processes. None of the process is progressing and they are all waiting. The computer becomes unresponsive since all the processes got blocked.



2 Necessary conditions for Deadlocks

1. Mutual Exclusion

A resource can only be shared in mutually exclusive manner. It implies, if two processes cannot use the same resource at the same time.

2. Hold and Wait

A process waits for some resources while holding another resource at the same time.

3. No preemption

The process which once scheduled will be executed till the completion. No other process can be scheduled by the scheduler meanwhile.

4. Circular Wait

All the processes must be waiting for the resources in a cyclic manner so thatthe last process is waiting for the resource which is being held by the first process.

3 Difference between Starvation and Deadlock

S.No.	Deadlock	Starvation
1	Deadlock is a situation where no process got blocked and no process proceeds	Starvation is a situation where the low priority process got blocked and thehigh priority processes proceed.
2	Deadlock is an infinite waiting.	Starvation is a long waitingbut not infinite.
3	Every Deadlock is always a starvation.	Every starvation need not be deadlock.
4	The requested resource is blocked by the other process.	The requested resource is continuously be used by the higher priority processes.
5	Deadlock happens when Mutual exclusion, hold and wait, No preemption and circular wait occurs simultaneously.	It occurs due to the uncontrolled priority and resource management.

4 Deadlock Prevention: We can prevent Deadlock by eliminating any of the above fourconditions.

- **Eliminate Mutual Exclusion**
  It is not possible to dissatisfy the mutual exclusion because some resources, such as the tape drive and printer, are inherently non-shareable.

- **Eliminate Hold and wait**

1. Allocate all required resources to the process before the start of its execution, this way hold and wait condition is eliminated but it will lead to low device utilization. For example, if a process requires printerat a later time and we have allocated printer before the start of its execution printer will remain blocked till it has completed its execution.

2. The process will make a new request for resources after releasing the current set of resources. This solution may lead to starvation.

- **Eliminate No Preemption:** Preempt resources from the process when resources required by other highpriority processes.

- **Eliminate Circular Wait:** Each resource will be assigned with a numerical number. A process canrequest the resources increasing/decreasing. Order of numbering.

  For Example, if P1 process is allocated R5 resources, now next time if P1 ask for R4, R3 lesser than R5 such request will not be granted, only request for resources more than R5 will be granted.

5 Deadlock Avoidance-

Deadlock avoidance can be done with Banker's Algorithm.

5.1 Banker's Algorithm

Banker's Algorithm is resource allocation and deadlock avoidance algorithm which test the entire request made by processes for resources,it checks for the safe state, if after granting request system remains inthe safe state it allows the request and if there is no safe state it doesn't allow the request made by the process**.**

Input to Banker Algorithm:

1. Maximum needs of resources by each process.
2. Currently allocated resources by each process.
3. Maximum free available resources in the system.

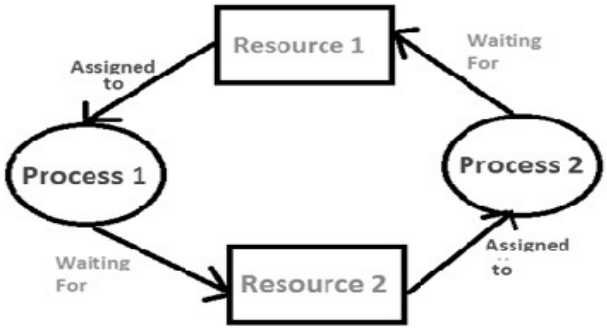
Request will only granted under below condition

1. Request made by process <= Maximum need of that process
2. Request made by process <= Freely available resources

6 Deadlock Detection:

1. If resources have a single instance –

In this case for Deadlock detection, we can run an algorithm to check for the cycle in the Resource Allocation Graph. The presence of a cycle in thegraph is a sufficient condition for deadlock.



In the above diagram, resource 1 and resource 2 have single instances. There is a cycle R1 → P1 → R2 → P2. So, Deadlock is Confirmed.

2. If there are multiple instances of resources –Detection of the cycle is necessary but not sufficient condition for deadlock detection, in this case, the system may or may not be in deadlock varies according to different situations.

7 Deadlock Recoveries:

A traditional operating system such as Windows doesn't deal with deadlock recovery as it is a time and space-consuming process. Real-time operating systems use Deadlock recovery.

- **Killing the process** –Killing all the processes involved in the deadlock. Killing process one by one. After killing each process check for deadlock again keep repeating the process till the system recovers from deadlock. Killing all the processes one by one helps a system to break circular wait condition.

- **Resource Preemption** – Resources are preempted from the processes involved in the deadlock, preempted resources are allocated to other processes so that there is a possibility of recovering the system from deadlock. In this case, the system goes into starvation

Unit related Questions

Q.1 What do you mean by Deadlock. Define its necessary conditions

Q.2 Differentiate between Deadlock and Starvation.

Q.3 Explain deadlock prevention in detail.

Q.4 What do you mean by Deadlock detection?

Q.5 Mention Deadlock recoveries in detail.